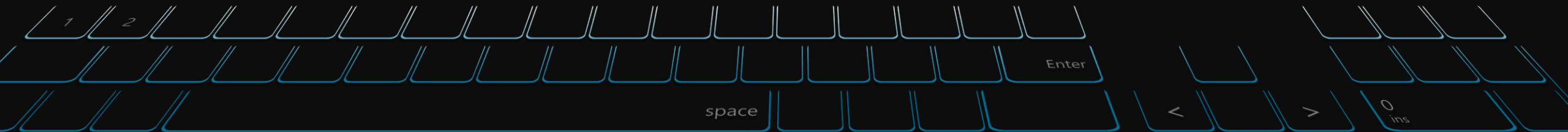


08 인공지능 & 회귀와 예측





강의 내용

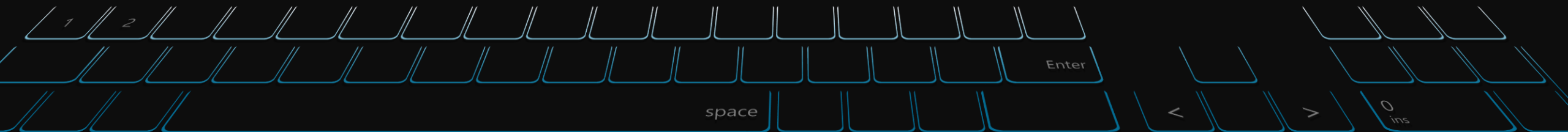
Contents of Lecture

기간	내용	과제
01주차	오리엔테이션 lpython - 파이썬에 날개를 달자	실습 과제 (채점 X)
02주차	Numpy 소개	실습 과제
03주차	Pandas로 데이터 가공하기	실습 과제
04주차	Matplotlib를 활용한 시각화	실습 과제
05주차	탐색적 데이터 분석	실습 과제
06주차	데이터의 표본분포	실습 과제
07주차	통계적 실험과 유의성 검정	실습 과제
08주차	중간고사 - 오프라인 오픈북으로 진행	기말 프로젝트 상세 공지

기간	내용	과제
09주차	인공지능 & 회귀/예측	실습 과제
10주차	회귀와 예측 / 분류	실습 과제
11주차	분류 및 머신러닝 기초	실습 과제
12주차	통계적 머신러닝	
13주차	비지도 학습	실습 과제
14주차	알고보면 쓸모있는 신기한 기계학습	프로젝트 레포트 제출
15주차	[기말고사] Team별 프로젝트 결과 발표	종강~!

1부: 인공지능

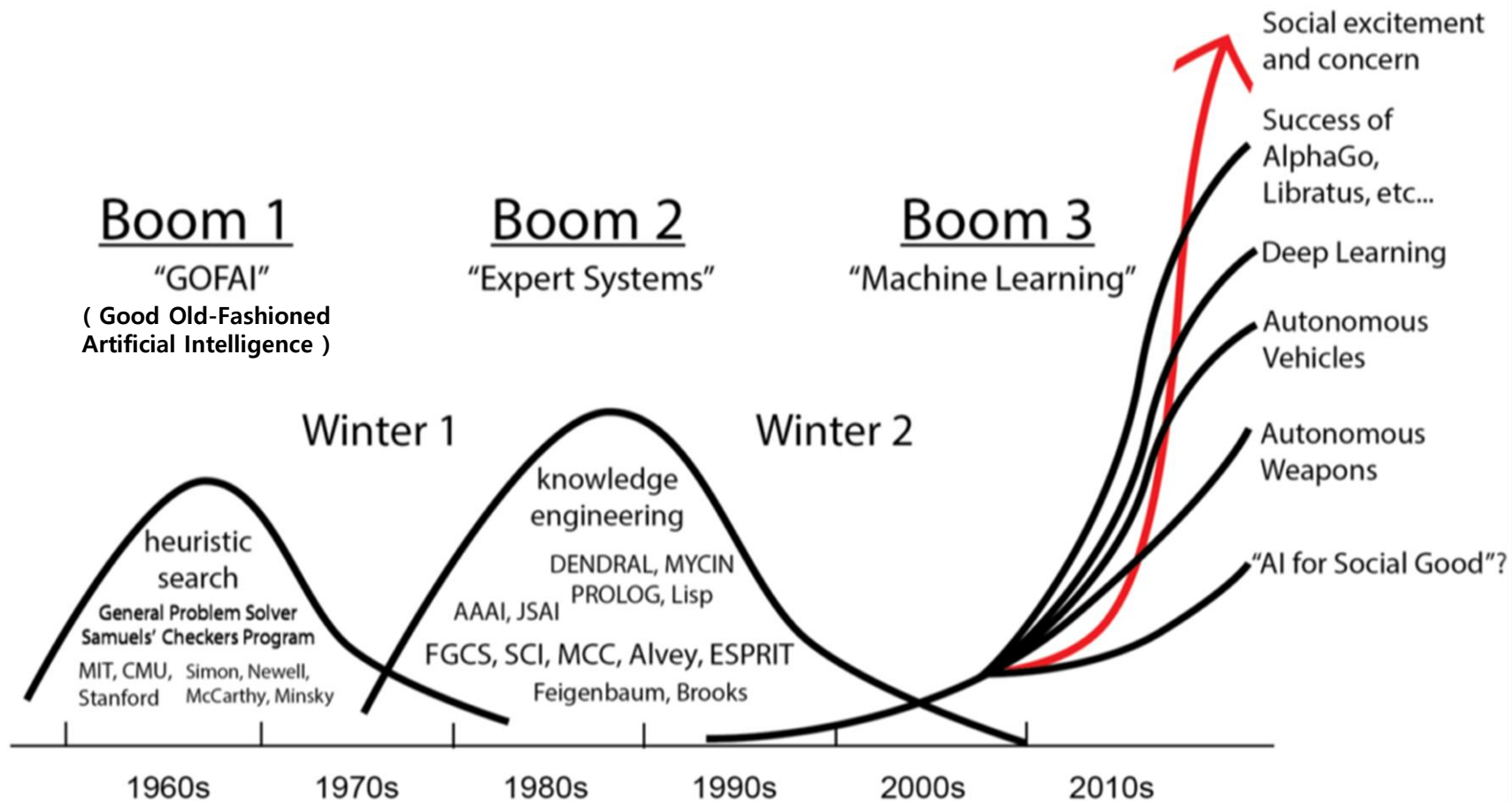
- 1) 인공지능 역사
- 2) 인공지능 정의
- 3) 인공지능 구분 및 사례
- 4) 모델링 프로세스
- 5) DevOps > MLOps > LLMOps





인공지능 역사

Artificial Intelligence History





인공지능 역사

Artificial Intelligence History

1672 : 파스칼 & 라이프니츠가 최초의 계산기 발명

1837 : 찰스 배비지, '명령들을 해석하면서 계산을 실행하는 기계' (Analytical Engine) 제작 시도

1950 : 앨런 튜링 '계산기계와 지성' 이라는 논문을 통해 인공지능에 대한 개념적인 토대 제시

1956 : 존 매커시, 다트머스 회의에서 AI (Artificial Intelligence) 용어 첫 등장 및 개념 정의

1957 : 프랭크 로센블라트, Perceptrons (인공신경망) 구현

1959 : 아서 사무엘, Machine Learning 정의

1969 : 마빈 민스키 & 사무엘 페이퍼트, Perceptrons 한계 지적

1st Winter of AI

1984 : 도그레나 교수, 상식능력을 가진 초기 인공지능 'Cyc' 발표

1986 : 데이빗 루멜하트, 오류역전파 재발견 (인공신경망) 귀환

1989 : 쉬평승 교수, 머리캠벨과 IBM 체스 특화 인공지능 '딥블루' 제작

1989 : 얀 레쿤, CNN 창시

2nd Winter of AI

1997 : IBM '딥블루' 체스 챔피언과의 대국에서 승리

2006 : 제프리 힌튼, Deep Belief Networks (딥러닝 시작)

2011 : IBM '왓슨' 퀴즈쇼 우승

2012 : DNN 시작, Neural Network가 이미지 인식대회에서 우승

2014 : Google, Tesla 초기버전의 자율주행자동차 기술 발표

2014 : 페이스북 얀 레쿤, '딥페이스' 알고리즘 개발

2016 : 구글 DeepMind 'AlphaGo' 바둑 챔피언 이세돌 상대 승리



여러분들의 기억 속 인공지능은 어떤가요?

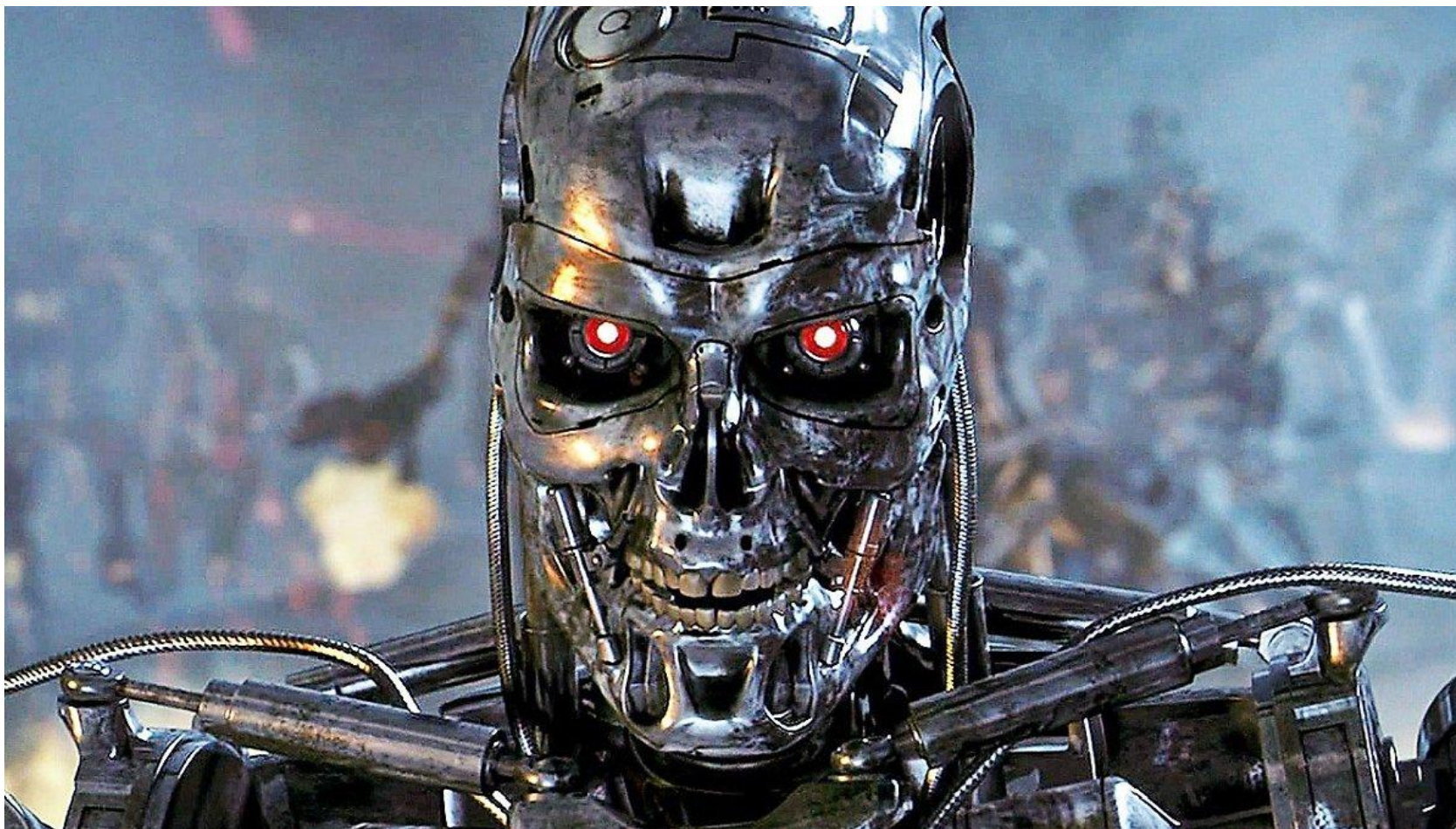
- 인간보다 **빠르고 정확한 두뇌**
- 인간보다 **우월한 신체 능력**
- 빅 데이터(Big Data)라는 정보의 홍수 속
탁월한 정보 선별능력 및 명확한 의사결정
- **실제 세계와 거의 동일한 가상 환경 생성**
- **과부하 없이, 스스로 강화학습 (자율학습)**
- 인간의 '**사랑**'이란 **감정** 보유



인공지능 정의

Definition of Artificial Intelligence

기억 속 인공지능 1 - 터미네이터

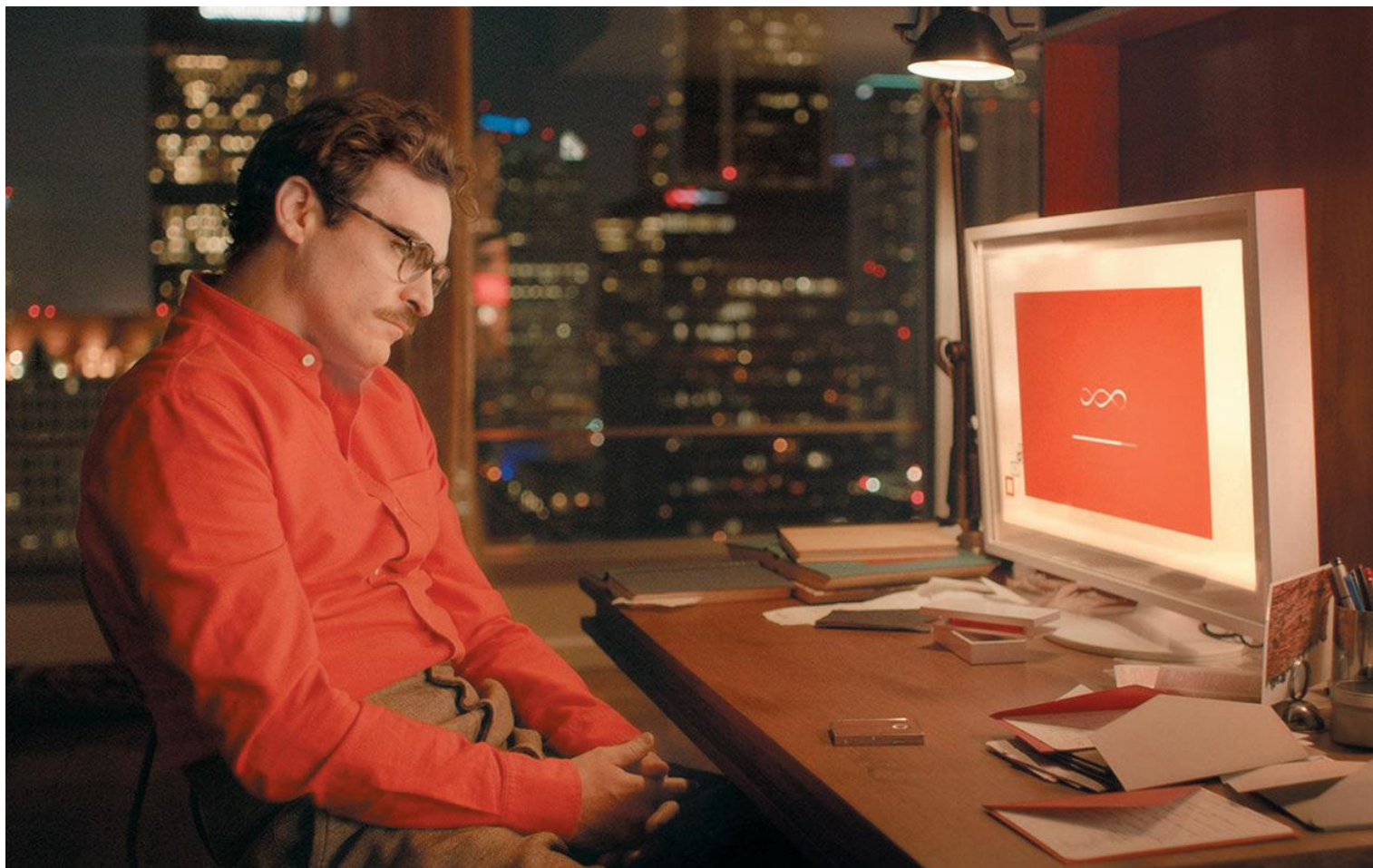




인공지능 정의

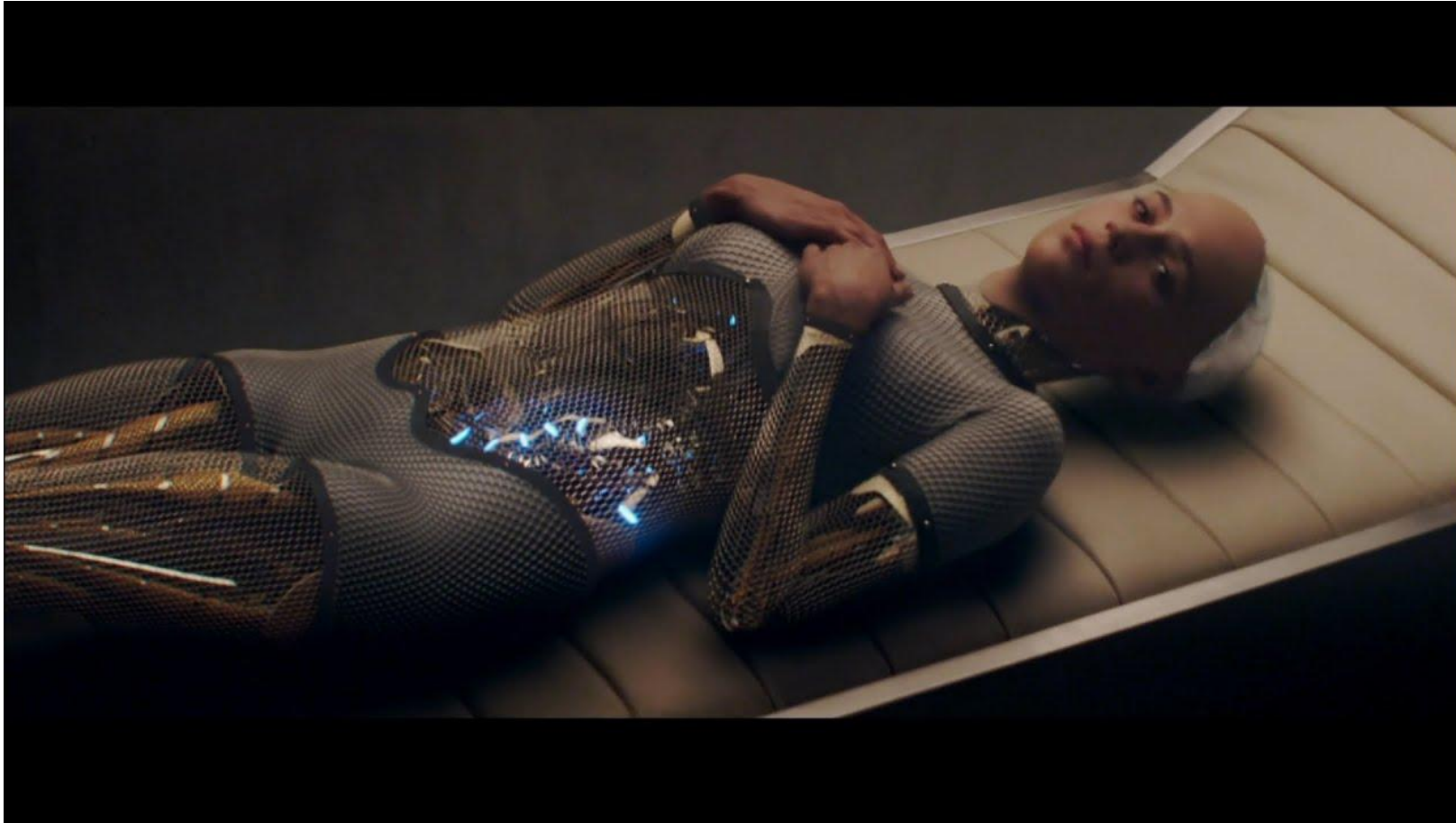
Definition of Artificial Intelligence

기억 속 인공지능 2 - HER





기억 속 인공지능 3 - 엑스마키나





실생활에서 본 인공지능은 어떤가요?

- **단일 Model, 그 자체**
(알고리즘 수준)
- **인간에 의한 구조 / 체계 디자인**
(스스로 생성 불가)
- **한계성 있는 환경 내에서 모형 구축**
(실제 환경과 동일한 수준으로 수치적인 표현가능할 때, 모형 구축 가능)
- **메모리 / 디스크 성능에 따라 과부하 존재**
(주어진 성능 내에서 학습 가능)



생활 속 인공지능 1 – AlphaGo (Deep Reinforcement Learning)





인공지능 정의

Definition of Artificial Intelligence

생활 속 인공지능 2 – Tesla (Computer Vision)





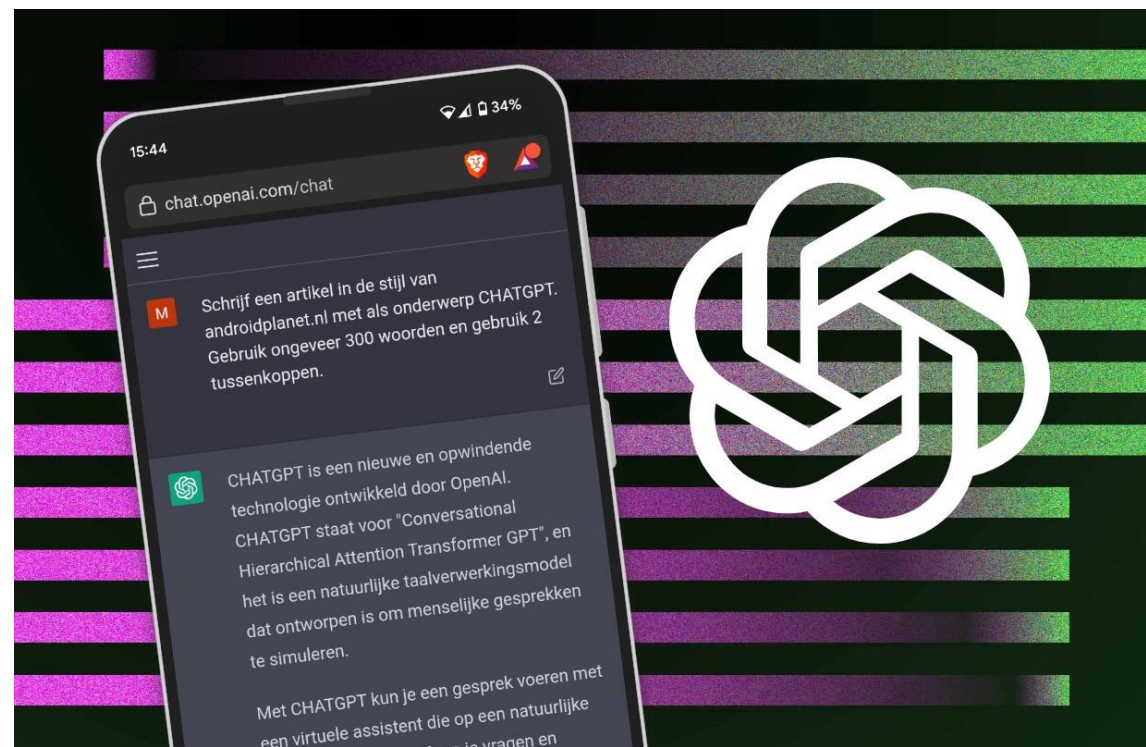
인공지능 정의

Definition of Artificial Intelligence

생활 속 인공지능 3 – Clova AI & ChatGPT (음성 인식 / 자연어 처리)



Clova AI Speaker
Friends Brown & Sally





현재, 인공지능의 정의는?

- 인간의 학습능력, 추론능력, 지각능력 등 그 외에 인공적으로 구현한 컴퓨터 프로그램 혹은 이를 포함한 컴퓨터 시스템
- 하나의 인프라 기술
- 인간을 포함한 동물이 가지고 있는 지능인 **Natural Intelligence(자연 지능)**과는 다른 개념
- 지능을 만들 수 있는 방법론 또는 실현 가능성을 연구하는 과학분야를 지칭하기도 한다



인공지능? 구분의 시작

Artificial Intelligence



인공지능

인간의 지적 능력을 컴퓨터를 통해 구현하는 기술



머신러닝

컴퓨터가 데이터를 통해 스스로 학습하여 예측이나 판단을 제공하는 기술

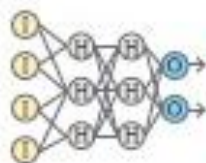
전문가
시스템

규칙기반
시스템

결정
트리

선형
회귀

퍼셉트론



딥러닝

깊은 인공신경망 알고리즘을 활용하는 머신러닝 기술

합성곱 신경망
(CNN)

심층
강화학습

순환 신경망
(RNN)



인공지능? 구분의 시작 => 머신러닝 VS 딥러닝

Artificial Intelligence

머신러닝 VS 딥러닝

- 전통적인 머신러닝
 - 주로 정형데이터 다룸
 - 관계형 데이터베이스 혹은 엑셀 표로 정리되는 테이블 형식의 데이터를 활용
 - 의사결정에 필요한 데이터를 사람이 최대한 정리한 후 기계에 알려주면 정보를 토대로 판단이나 예측
- 딥러닝
 - 주로 비정형데이터 다룸
 - 즉, 지정된 방식으로 정리되지 않은 정보를 활용
 - 예시) 이미지, 비디오, 텍스트 문장, 문서, 음성 데이터 등



인공지능? 구분의 시작 => 머신러닝 VS 딥러닝

Artificial Intelligence

머신러닝

VS

딥러닝

Structured
Data



정형데이터

Unstructured
Data



VS

비정형 데이터



지도 / 비지도 / 준지도 / 강화 학습

1st 인공지능 학습 기준 분류

분류되는 기준

1. 학습하는 동안 사람이 감독(Labeling)하는지 여부
2. 감독된(Labeling된) 정보량에 따른 분류

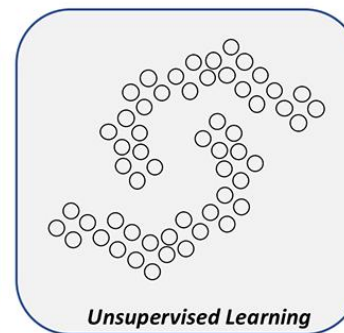
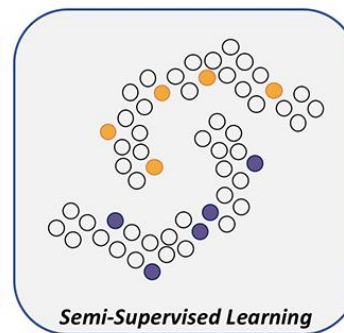
**SUPERVISED
LEARNING**



**UNSUPERVISED
LEARNING**



**REINFORCEMENT
LEARNING**



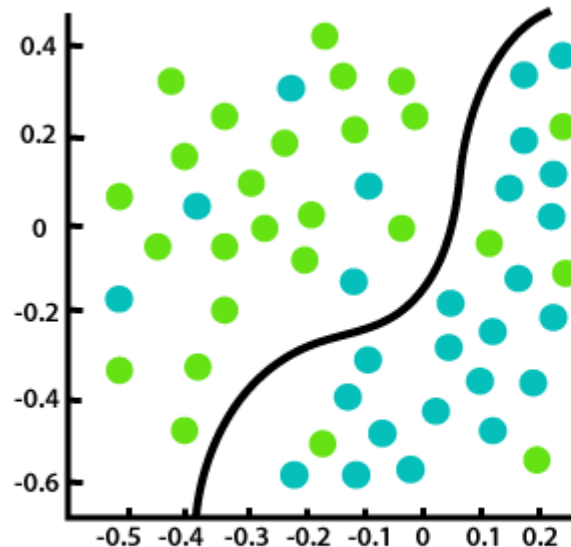


지도 / 비지도 / 준지도 / 강화 학습

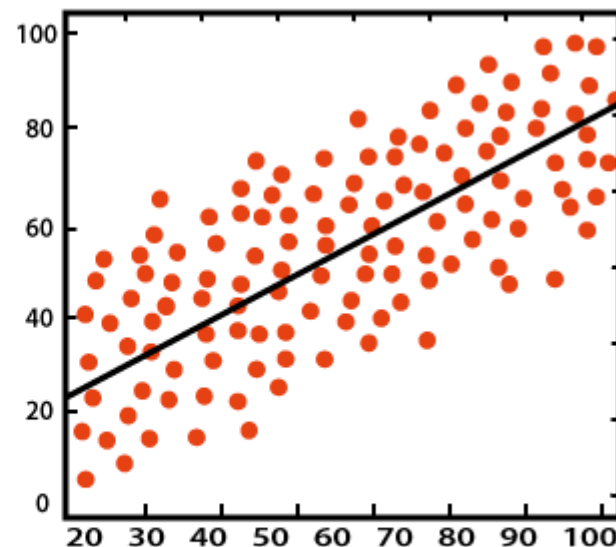
1st 인공지능 학습 기준 분류

지도 학습

- Labeling된 정답지 데이터가 존재
- 전형적인 지도 학습의 모형
 - 분류(Classification) & 회귀(Regression)



Classification



Regression



지도 / 비지도 / 준지도 / 강화 학습

1st 인공지능 학습 기준 분류

지도학습

- Labeling된 정답지 데이터가 존재
- 전형적인 지도 학습의 모형
 - 분류(Classification) & 회귀(Regression)

가장 중요한 지도학습 알고리즘

1. K-Nearest Neighbors (KNN)
2. Linear Regression
3. Logistic Regression
4. Support Vector Machines (SVMs)
5. Decision Tree / Random Forests
6. Neural Networks

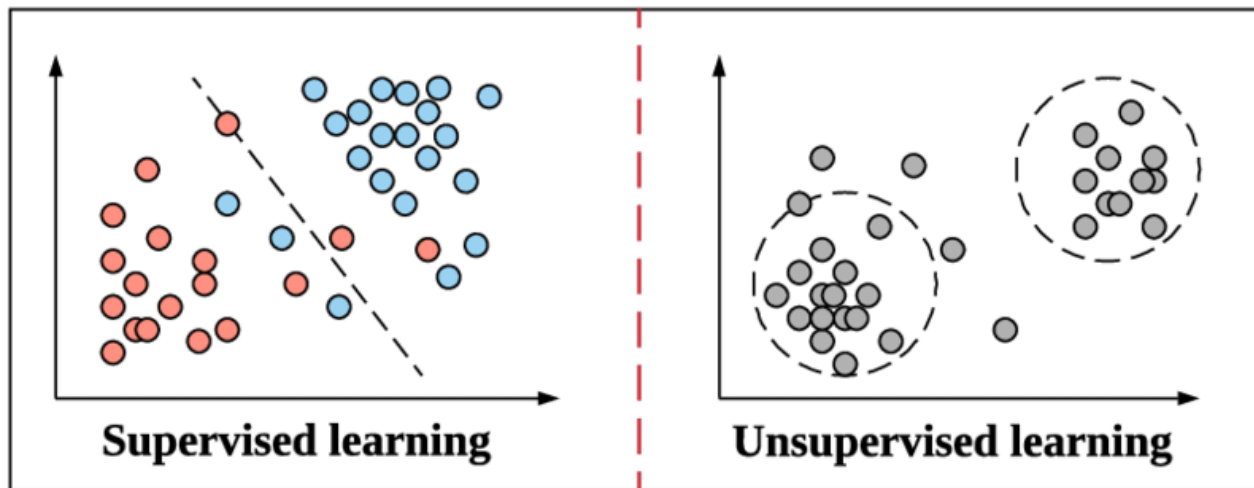


지도 / 비지도 / 준지도 / 강화 학습

1st 인공지능 학습 기준 분류

비지도 학습

- Labeling된 정답지 데이터가 존재하지 않음
- 전형적인 비지도 학습의 모형
 - 군집 (Clustering)
 - 시각화 (Visualization)
 - 차원축소 (Dimension Reduction)
 - 연관규칙학습 (Association Rule Learning)



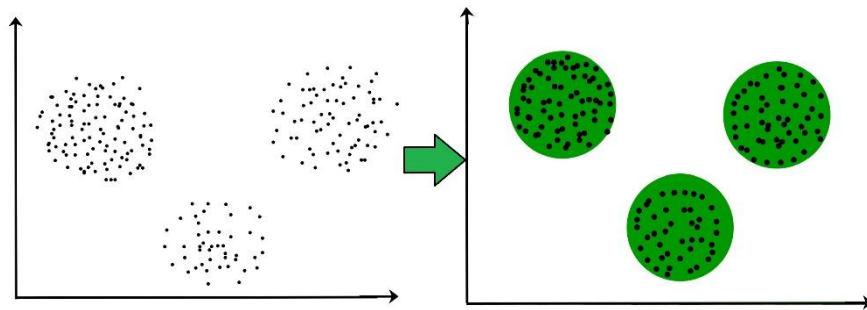


지도 / 비지도 / 준지도 / 강화 학습

1st 인공지능 학습 기준 분류

비지도학습 - 군집

- Labeling된 정답지 데이터가 존재하지 않음.
- 하지만, 데이터 특성을 기반으로 비슷한 그룹으로 세분화 진행



비지도학습 - 군집 알고리즘

1. K-평균 (K-Means)
2. DBSCAN
3. 계층 군집 분석 (Hierarchical Cluster Analysis : HCA)
4. 이상치 탐지 (Outlier Detection) 와
특이치 탐지 (Novelty Detection)
5. 원-클래스 (One-class SVM)
6. 아이솔레이션 포레스트 (Isolation Forest)

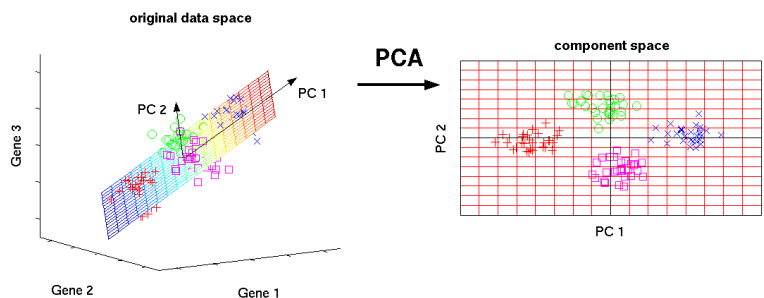


지도 / 비지도 / 준지도 / 강화 학습

1st 인공지능 학습 기준 분류

비지도학습 – 시각화 & 차원축소

- Labeling된 정답지 데이터가 존재하지 않음.
- 하지만, 데이터 특성을 기반 n차원 데이터-> 2차원 데이터로 보다 군집화가 잘 되도록 시각화 가능한 차원 축소 변환



비지도학습 – 시각화 & 차원축소 알고리즘

1. 주성분 분석 (Principal Component Analysis : PCA)
2. 커널 PCA (Kernel PCA)
3. 지역적 선형 임베딩 (Locally Linear Embedding : LLE)
4. T-SNE (t-distributed stochastic neighbor embedding)



지도 / 비지도 / 준지도 / 강화 학습

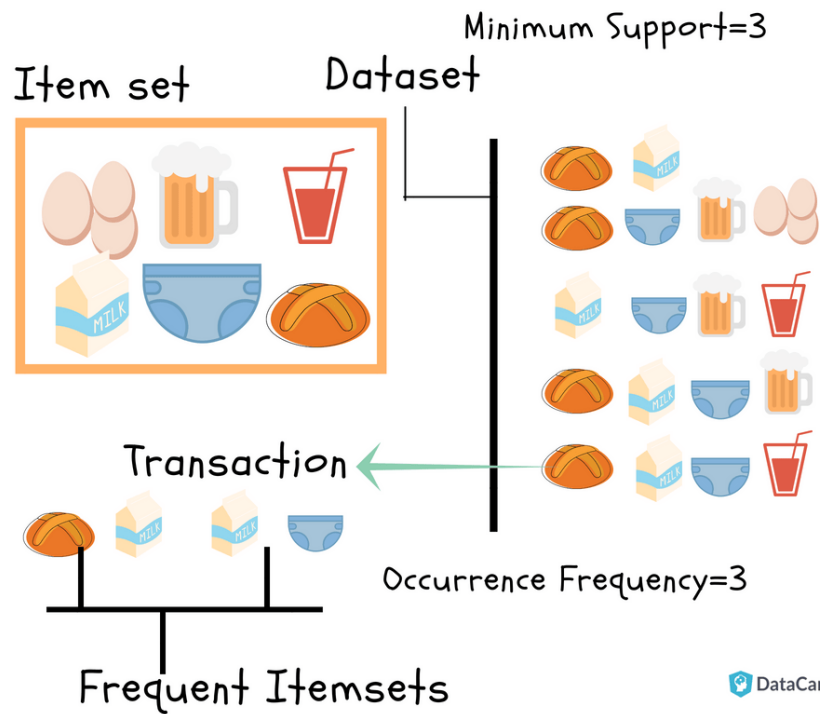
1st 인공지능 학습 기준 분류

비지도학습 - 연관 규칙 학습

- Labeling된 정답지 데이터가 존재하지 않음.
- 대량의 데이터에서 특성 간 흥미로운 관계를 찾는 학습 방법

비지도학습 - 연관 규칙 학습 알고리즘

1. 어프라이어리 Apriori
2. 이클렛 Eclat





지도 / 비지도 / 준지도 / 강화 학습

1st 인공지능 학습 기준 분류

준지도 학습

- 일반적으로 Labeling에 시간과 비용이 많이 들기 때문에 데이터 중 Labeling된 데이터가 적은 경우가 많을 때 활용되는 학습 방법
- 전형적인 준지도 학습의 모형
 - 심층 신뢰 신경망 (Deep Belief Networks : DBN)
 - > 제한된 볼츠만 머신 (Restricted Boltzmann Machine : RBM)이란 비지도학습 기반으로 순차적 학습 이후, 지도학습 방식으로 세밀하게 조정하는 모형

그림 1-11 준지도 학습





지도 / 비지도 / 준지도 / 강화 학습

1st 인공지능 학습 기준 분류

강화학습

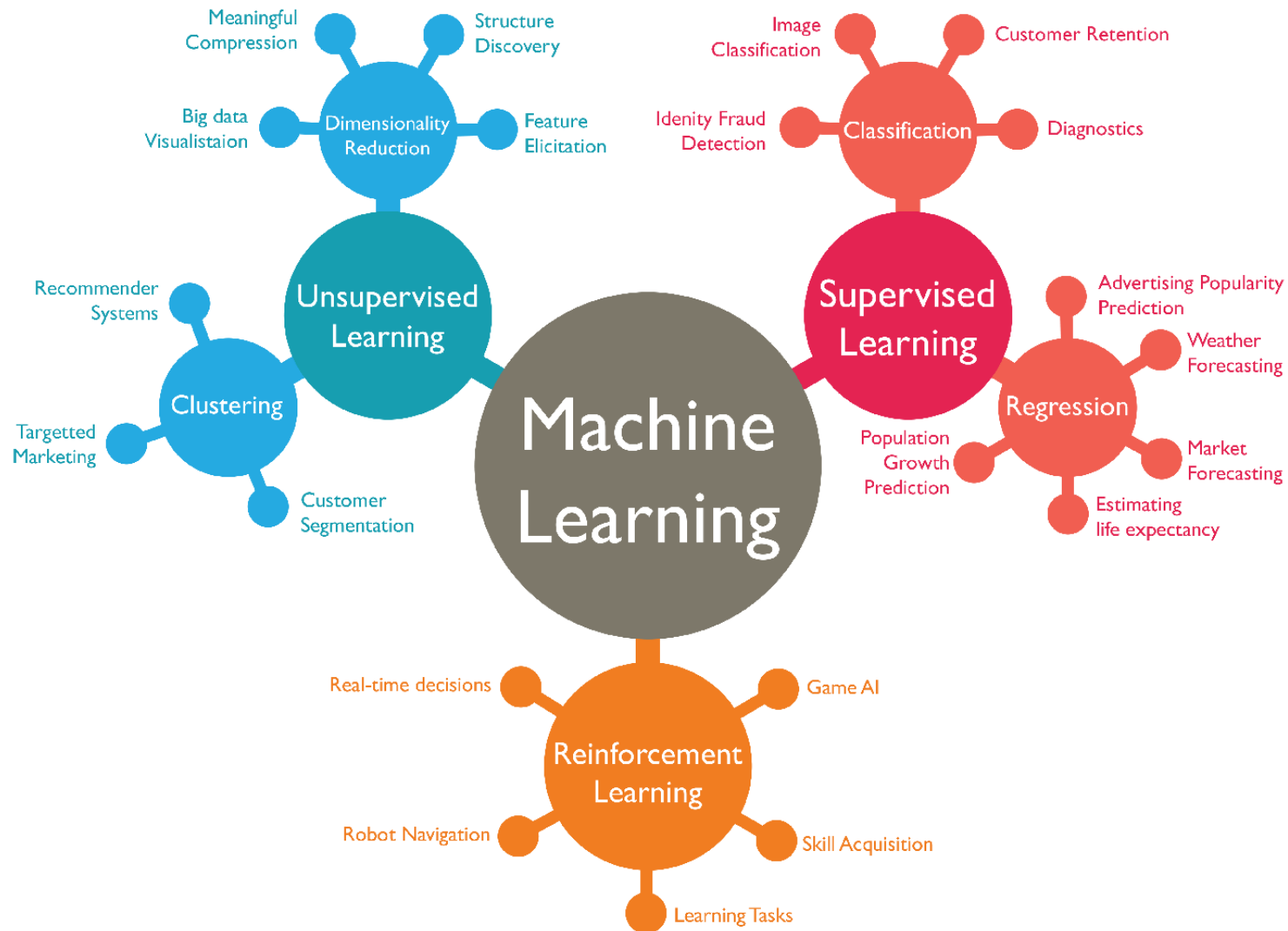
- 지도/비지도/준지도 학습과 매우 다른 종류의 알고리즘
- 에이전트(Agent)가 환경(Environment)를 관찰해서 행동(Action)하고 그 결과로 보상(Reward) 혹은 벌점(Penalty)을 얻어서, 가장 큰 보상을 얻기위한 최상의 전략인 정책(Policy)을 스스로 학습하는 방법
- 예시
 - 보행 로봇
 - 딥마인드, 알파고 (Deepmind, AlphaGo)





지도 / 비지도 / 준지도 / 강화 학습

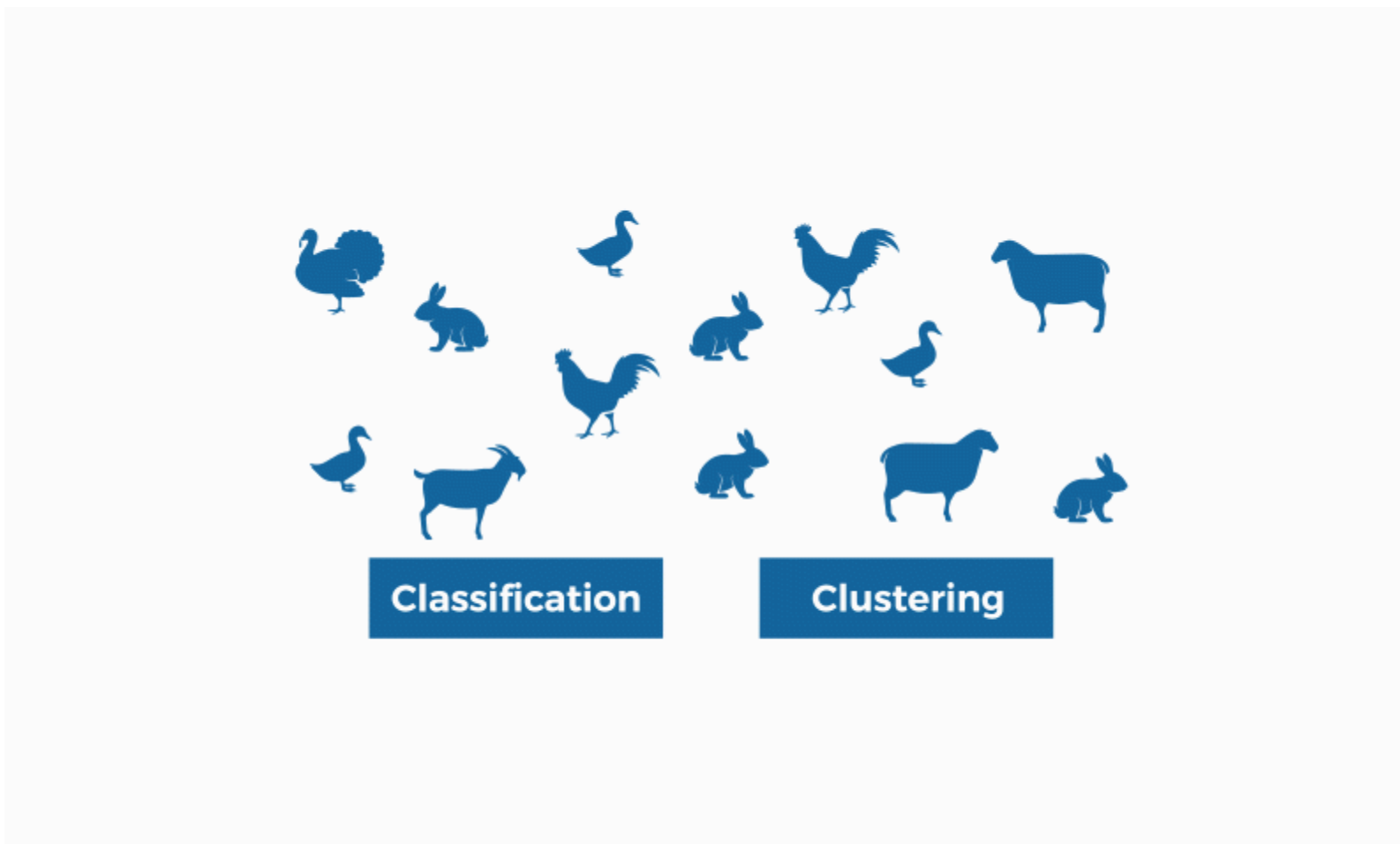
1st 인공지능 학습 기준 분류





지도 / 비지도 / 준지도 / 강화 학습

분류 vs 군집



[Classification Vs. Clustering - A Practical Explanation](#)



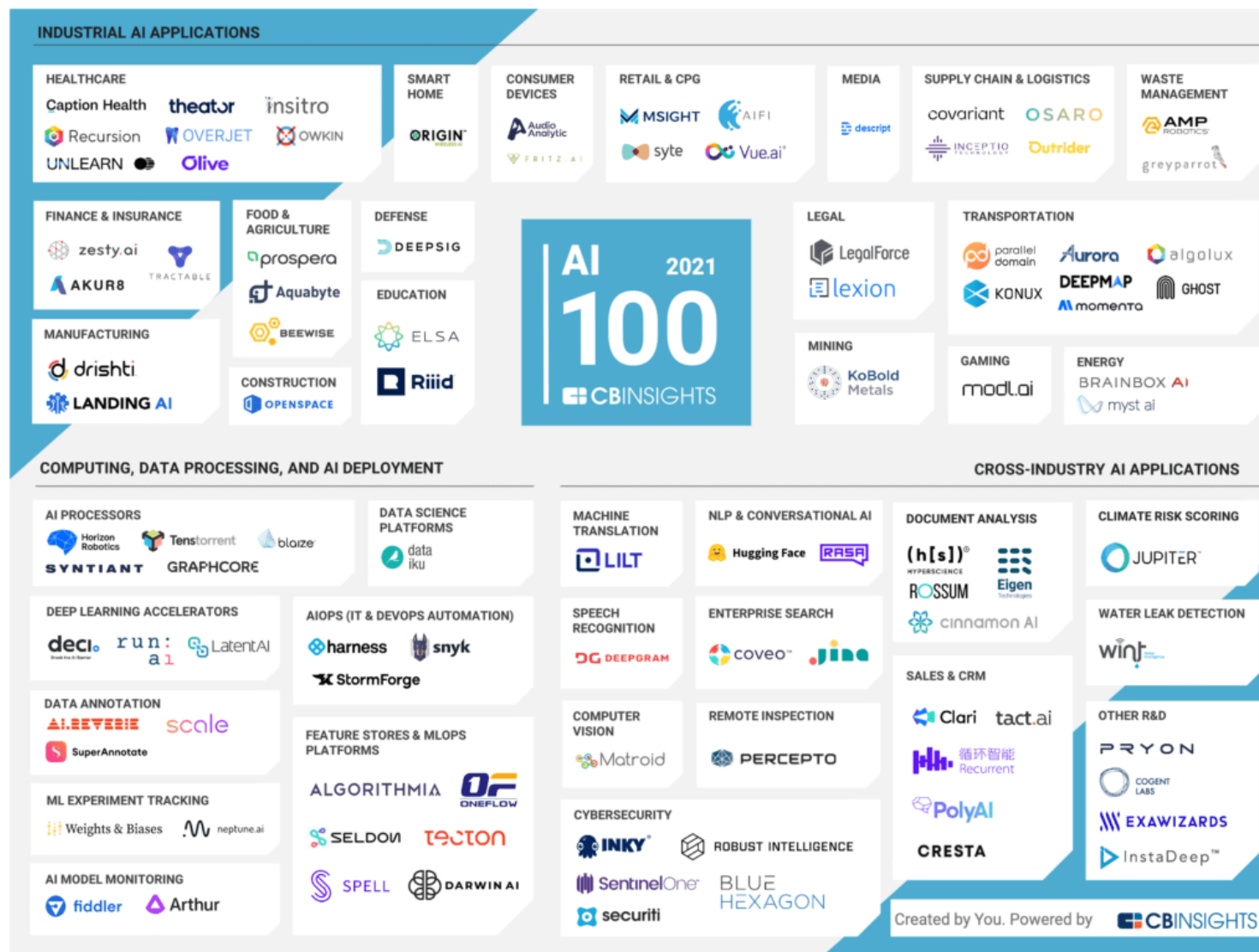
[ML | Classification vs Clustering](#)

[Data Mining 문제 및 답안](#)

[40 Questions to test a Data Scientist on Clustering Techniques \(Skill test Solution\)](#)



글로벌 비즈니스에서의 인공지능 (2021)





글로벌 비즈니스에서의 인공지능 (2022)



Cross-industry applications

Manufacturing



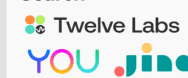
Warehouse automation



Sales & contact center



Search



Cybersecurity



Customer feedback analysis



Location data



Worker safety & incident prevention



Business intelligence



Engineering design



IT & devops automation

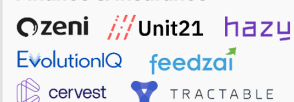


Other R&D



Industry-specific applications

Finance & insurance



Retail



Healthcare



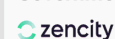
Telecom



Aerospace & defense



Government



Auto



Agriculture



Construction



Maritime



Gaming



Waste management



Media



AI development tools

AI chips



Data annotation



Synthetic data



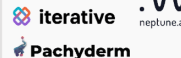
Data de-identification



Data quality & observability



Version control & experiment tracking



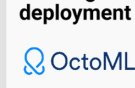
Model validation & monitoring



ML platforms



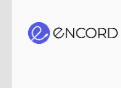
Machine learning deployment



Resource optimization



Computer vision



Natural language processing





글로벌 비즈니스에서의 인공지능 (2023)



AI development tools

AI chips & processors 	Synthetic data 	NLP annotation 	Computer vision visualization 	Federated learning platforms 	Version control & experiment tracking
Vector database tech 	ML development & deployment 	Model validation & monitoring 	AI auditing & governance 	Foundational models & APIs 	

Cross-industry applications

Synthetic voice 	Image & text generation 	Privacy & security 	Code generation 	Sales & customer support 	AI assistants & HMs
Emotion analytics 			Digital twins 		Design tools
Productivity tools 	Warehouse & logistics 	Content moderation 	Smell tech 	Translation 	Climate tech
				Quantum AI software 	
General-purpose humanoids 	Image processing 	Search 			

Industry-specific

Materials & manufacturing  Cradle  MakinaRocks	Gaming  convai  MODULATE	Fashion & retail refiberd  see1	Energy  phaidra  WeaveGrid	Healthcare  AQEMIA  Atomic AI  ATROPOSHEALTH  Generate:Biomedicines  navina  SUBTLE MEDICAL  turbine.  XtalPi
Defense  VANNEVAR Labs	Finance  arteria  Boosted.ai	Agriculture  BEEWISE  VERDANT ROBOTICS	Physical infrastructure  exadigo	
Education  ello	Media & entertainment  character.ai  descript  FLANER  MO  runway  wonder	Legal  Harvey.	Auto & mobility  helm.ai  PHANTOM AI	Construction  Augmenta



글로벌 비즈니스에서의 인공지능 (2023-11-03)

AI-powered automation solutions for retail banking leaders

Cross-functional RPA



Customer experience tools



KYC & customer onboarding automation



Loan & mortgage process automation



Process discovery & mining



Intelligent document processing





국내 비즈니스에서의 인공지능 (2020)

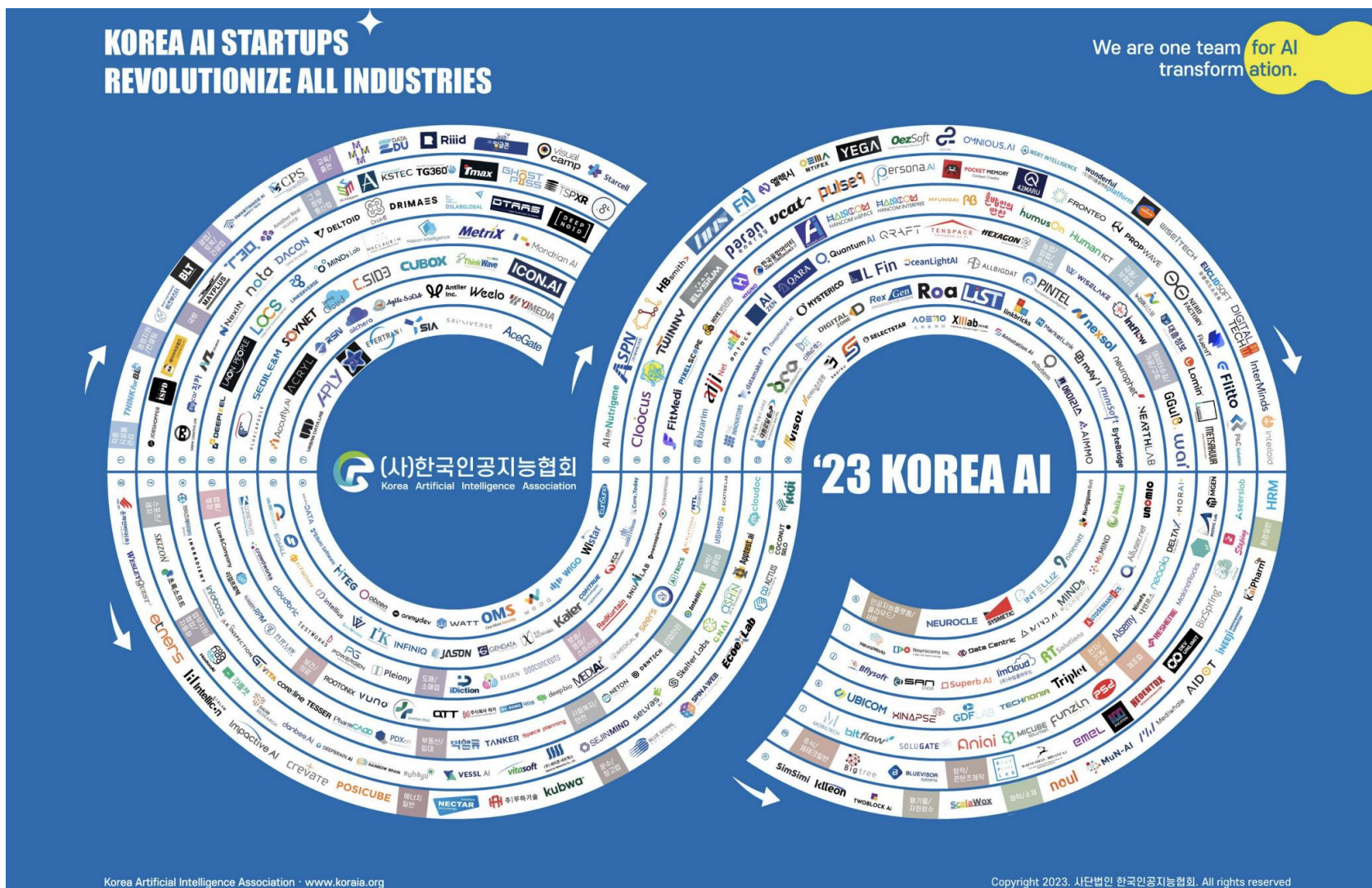
사단법인 한국인공지능협회

200 KOREA AI STARTUPS REVOLUTIONIZE ALL INDUSTRIES

AI PLATFORM Skelter Labs, persona, Mondrian AI, Superb AI, WIGO, ImCloud, SOLIDWARE, acryl, ELGEN, MINDa Lab, bpu	TRAVEL/TRANSLATION AccuFly AI, EVERTRAN, Flitto, HANCOM
FUTURE CAR MOBILTECH, MORAI, CODE42	AI CHIP/AI TESTING TrueFarm AI, FURIOSA, STA
DRONE APLY, SAFEUS DRONE, MAYPLUS	RETAIL-TECH AISTUDIO, HINE, TXINNY, hanvit
INTELLIGENT ROBOT MACROACT, wonderful	SAFETY&SECURITY ellexi, DBNtech, EUCLIDSOFT, YEGA, SPINA WEB, COMTRUE, bitflow
NEW MATERIAL MTIFEX	ADVERTISING/SALES MARKETING seerslab, VOICE camp, LOCS, LSPB, CSID3
BIO PDX, noul, KaiPharm, Infoboss, 3billion, Standigm, PharmCARE	INSUR-TECH AI ZEN, Lomin
MEDICAL NEUROPHET, JLK INSPECTION, ELYSIUM, AI TRICS, CORELINE, BUZZPOLE, Medi Whole, VUNO, baikal.ai, Lunit	FASHION/BEAUTY DEEPIKEL, URBAN UNION, OMNIOUS, POKET MEMORY, Styling
HEALTH & WELFARE Codevision, PIXEL, Mr. Mind, DEEPLY, FIT CARE, ALETHIO	FOOD/SPORTS MYSTERICO, PIXELSCOPE, doing lab, IDEIDY
HOME APPLIANCE ICONAI, SCATTER LAB, 42MARU, cochlear.ai, BRYDGE AI	ART/CONTENTS NEWPLUS, pulse9, POZA, MONEYBRAIN
SMART MANUFACTURING RAINBOW BRAIN, IND, LAON PEOPLE, PEC, Vitalize Lab	MEDIA/ENTERTAINMENT XINAPSE, SRUNIVERSE, NEOSAPIENCE
SMART CITY NETON, CONTINUUM, ScalaWox, Aichera, BLUE SIGNAL, CTSens	NATIONAL DEFENSE/AEROSPACE SIA, InSpace
SMART ENERGY ninewatt, paran, MAKINA ROCKS	E-COMMERCE 0.8L, SOYNET, Stratio, Nalbi
SMART ANIMAL HUSBANDRY & AGRICULTURE INT-FLOW, INT-TECH	ON DEVICE/IOT Neurocoms Inc., NEUROCLE, nota, SOYNET, Stratio, Nalbi
FIN-TECH QRAFT, OezSoft, Bigtree, QARA, BV, Nomad Connection, DeepSearch	CONTACT CENTER HBsmith, Gitple, danbee AI, CnAI, ATLAS labs
EDU-TECH Riidi!, Starcell, SimSimi, crevate, elice, 2NEU, JOBSHOPPER, MATHPRESSO	DATA MARKET INFINIQ, KUBWA, RIMMO, crowdworks, SELECT STAR, TESTWORKS, DATA INSURANCE KOREA
REAL ESTATE TANKER, 박한규, 다들랜	ENTERPRISE & PUBLIC SERVICE T30, DEEPSpace, WISEITECH, SOLUGATE, NERD FACTORY, Ingris
LAW/PATENT LAW/TALK, 인텔리콘, SYSTRAN, SEJINMIND, WERT, HELP ME	GENERAL SERVICE XONO, Story Brothers, Alliance Korea, VOYABERS, Spirink, 마이셀럽스, moditt



국내 비즈니스에서의 인공지능 (2023)





국내 비즈니스에서의 인공지능 (2023)





국내 AI STARTUP Top 100 (2021)





국내 AI STARTUP Top 100 (2022)

INDUSTRIAL AI APPLICATIONS

개별산업



CROSS- INDUSTRY AI APPLICATIONS

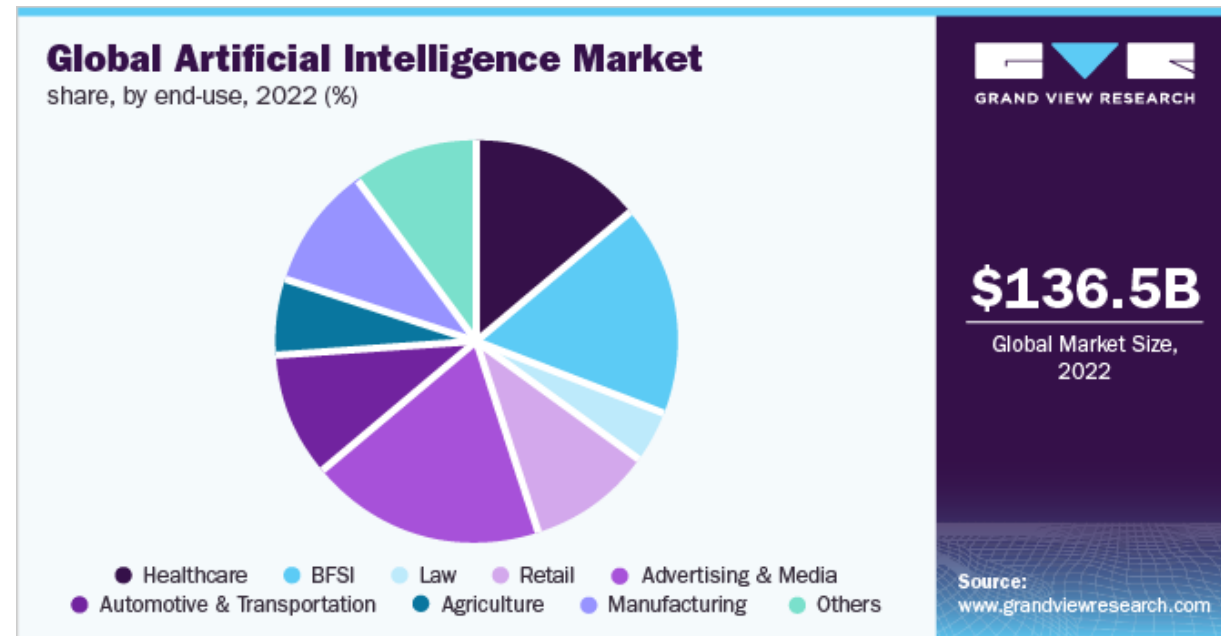
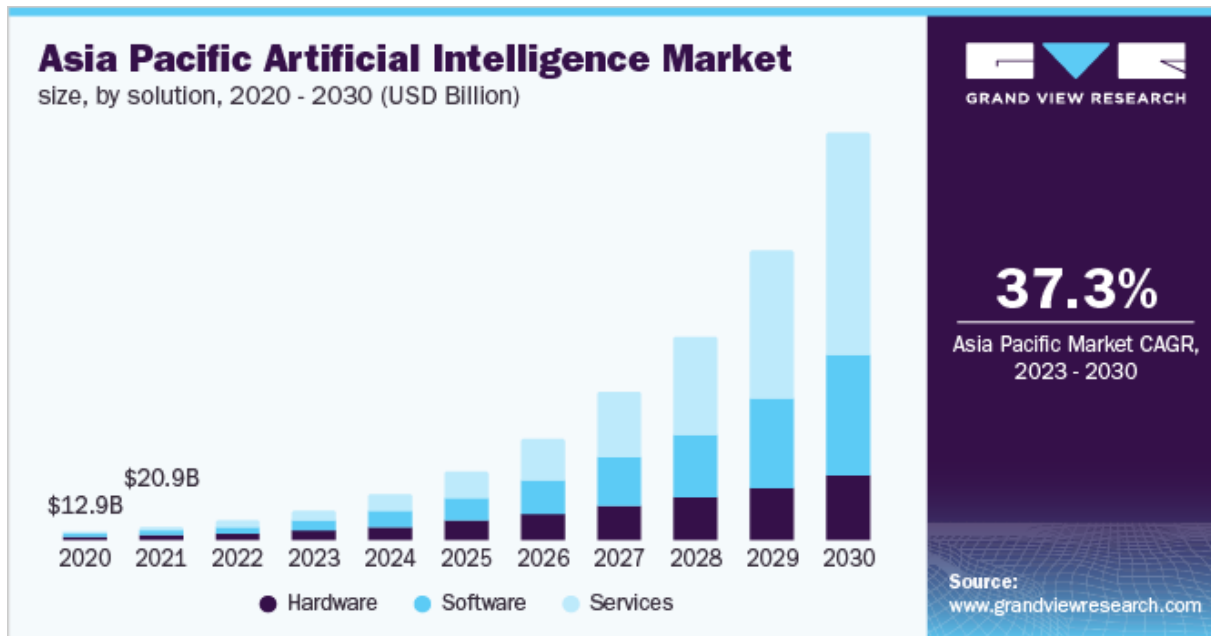
산업공통





인공지능 비즈니스의 가치

<https://www.grandviewresearch.com/industry-analysis/artificial-intelligence-ai-market>



2023년 글로벌 AI 시장 : USD 196.63 billion in 2023 (약 263조 가치)

2023-2030년 예상되는 연간 성장률 : 37.3%

(USD 1,811.75 billion in 2030, 약 2,427조 가치 예상)

* 반도체 시장과의 비교, 2021년 기준, 글로벌 반도체 시장의 미래 연간 성장률은 약 4-10% 정도로 예측됨



스마트 시티 (Computer Vision / Image Recognition)

사진 및 영상 기반, 얼굴 검출 및 인식 기술

- 카메라에 나온 것이 진짜 사람의 얼굴인지 Liveness 인식
- 사전 등록된 개인의 얼굴 인식
- 마스크 착용 여부 / 연령 / 성별 / 표정 추정

Anomaly Analysis

- Computer Vision 기반, 화재검출 시스템 (연기 감지 / 화재 감지)

Behavior Analysis

- 실시간 사람 추적 기능
- 영상기반 동선 파악 시스템
- 고객 안전 모니터링

인공지능 비즈니스 초기 구축사례 확인 방법

1. 산업별 기업 편람 탐색(SW, 핀테크, AI 등)
2. 비즈니스 및 성장 가능성 파악
3. 특정 회사의 상세한 내용을 알고 싶다면 Linkedin으로 해당 회사의 시니어 직원에게 직접 연락해서 확인!



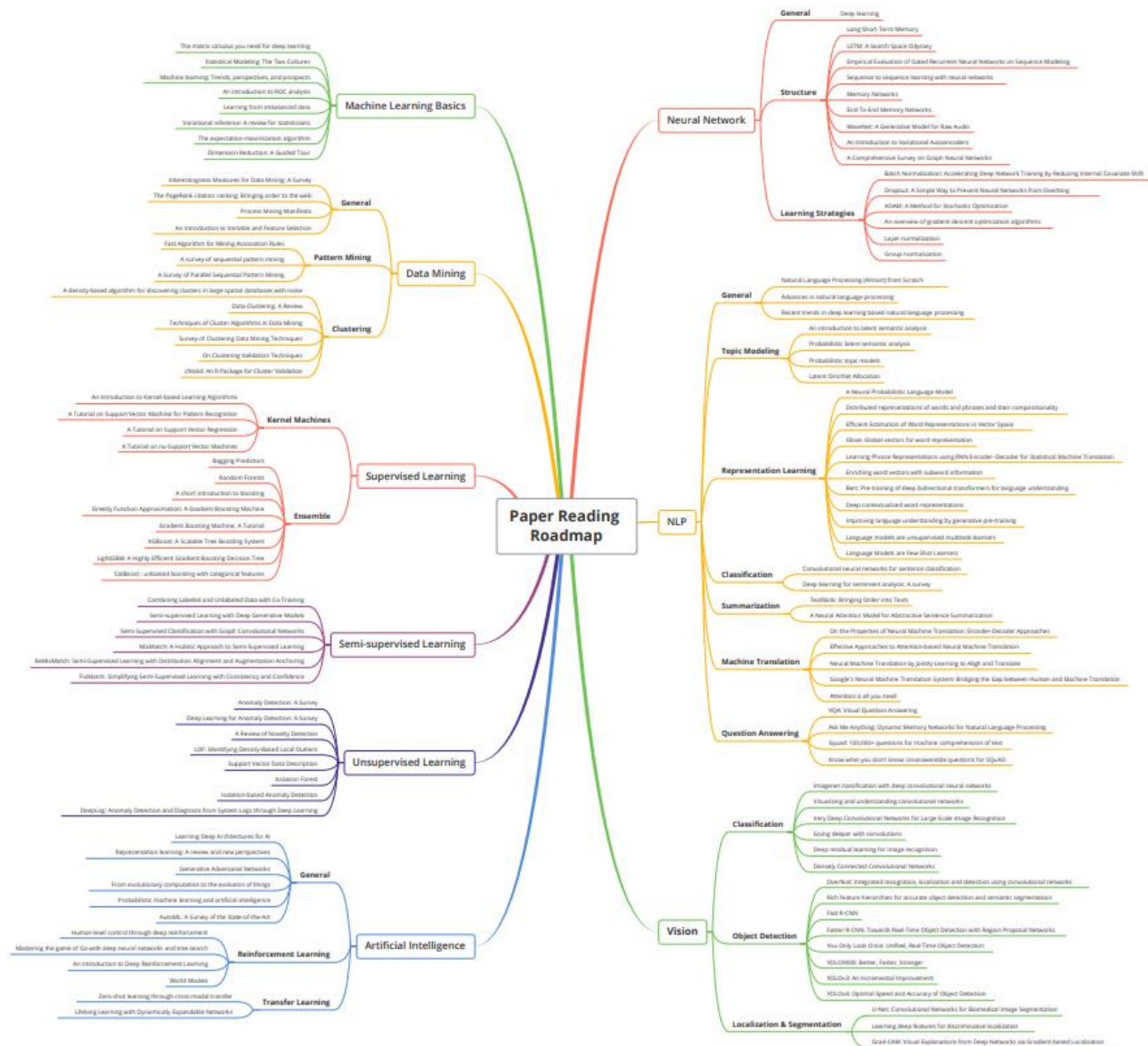


인공지능 모델 대표 분야

Main Fields of Artificial Intelligence

Data Science & Business Analytics 구분

- Machine Learning Basic
- Data Mining
- Supervised Learning
- Semi-Supervised Learning
- Unsupervised Learning
- Artificial Intelligence
- Neural Network
- NLP (Natural Language Processing)
- Computer Vision





인공지능 모델 대표 사례

Main Fields of Artificial Intelligence

텍스트

- 자동으로 뉴스 기사를 분류
 - 텍스트 분류 작업, 자연어 처리(natural language processing; NLP) 작업
 - 순환 신경망(recurrent neural network; RNN), 합성곱 신경망(convolutional neural network; CNN), 트랜스포머(Transformer)를 사용해 해결
- 토론 포럼에서 부정적인 코멘트를 자동 구분하기
 - 텍스트 분류 작업
 - 자연어 처리(natural language processing; NLP) 도구 사용
- 긴 문서를 자동 요약
 - 텍스트 요약
 - 자연어 처리(natural language processing; NLP) 도구 사용
- 챗봇 또는 개인 비서 만들기
 - 자연어 이해(natural language understanding; NLU)와 질문-대답(question-answering) 모듈을 포함해 여러가지 자연어 처리(natural language processing; NLP) 컴포넌트 필요



인공지능 모델 대표 사례

Main Fields of Artificial Intelligence

음성/영상

- 생산라인에서 제품 이미지를 분석해 자동 분류
 - 이미지 분류 작업
 - 일반적으로 합성곱 신경망(convolutional neural network; CNN)을 사용하여 수행
- 뇌를 스캔하여 종양 진단
 - 시맨틱 분할 작업
 - 일반적으로 합성곱 신경망(convolutional neural network; CNN)을 사용하여 이미지의 각 픽셀을 분류
- 음성 명령에 반응하는 앱 만들기
 - 음성 인식 작업
 - 순환 신경망(recurrent neural network; RNN), 합성곱 신경망(convolutional neural network; CNN), 트랜스포머(Transformer)를 사용



인공지능 모델 대표 사례

Main Fields of Artificial Intelligence

기타

- 다양한 성능 지표를 기반으로 회사의 내년도 수익 예측
 - 회귀 작업(숫자로 값을 예측)
 - 선형 회귀(linear regression)나 다항 회귀(polynomial regression) 모델, 회귀 SVM(support vector machine), 회귀 랜덤 포레스트(random forest), 인공신경망(artificial neural network)과 같은 회귀 모델을 사용해서 해결
 - 지난 성능 지표의 시퀀스를 고려한다면 한다면 순환 신경망(recurrent neural network; RNN), 합성곱 신경망(convolutional neural network; CNN), 트랜스포머(Transformer)를 사용
- 신용 카드 부정 거래 감지
 - 이상치 탐지 작업
- 구매 이력을 기반으로 고객을 나누고 각 집합마다 다른 마케팅 전략 계획하기
 - 군집(clustering)
- 고차원의 복잡한 데이터셋을 명확하고 의미 있는 그래프로 표현
 - 데이터 시각화 작업
 - 차원축소(dimensionality reduction) 기법 많이 사용
- 과거 구매 이력을 기반으로 고객이 관심을 가질 수 있는 상품 추천
 - 추천 시스템
 - 데이터를 인공 신경망에 주입하고 그 결과물로 상품 추천
- 지능형 게임 봇 만들기
 - 보통 강화 학습(reinforcement learning; RL)으로 해결



인공지능 모델 대표 사례

Main Fields of Artificial Intelligence

Computer Vision 분류

1. Image Classification
2. Face Recognition
3. Neural Style Transfer Using GANs
4. Scene Text Detection
5. Object Detection With DETR
6. Semantic Segmentation
7. Road Lane Detection in Autonomous Vehicles
8. Image captioning
9. Human Pose Estimation Projects
10. Emotion Recognition through Facial Expressions



인공지능 모델 대표 사례

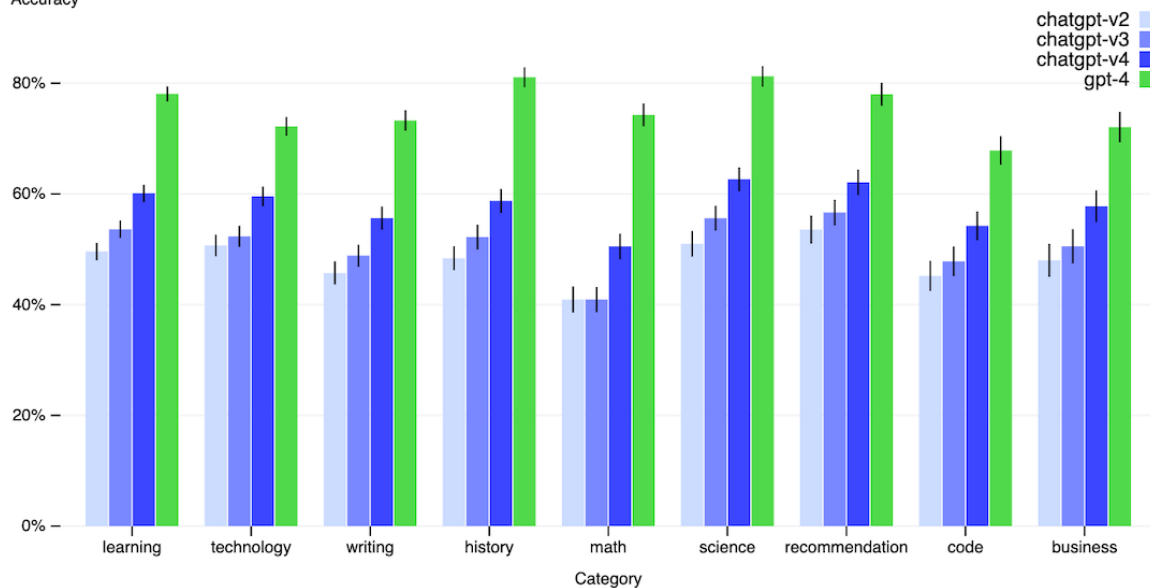
Main Fields of Artificial Intelligence

LLM(Large Language Model)

• GPT 4.0 분야별 & 시험별 성능

Internal factual eval by category

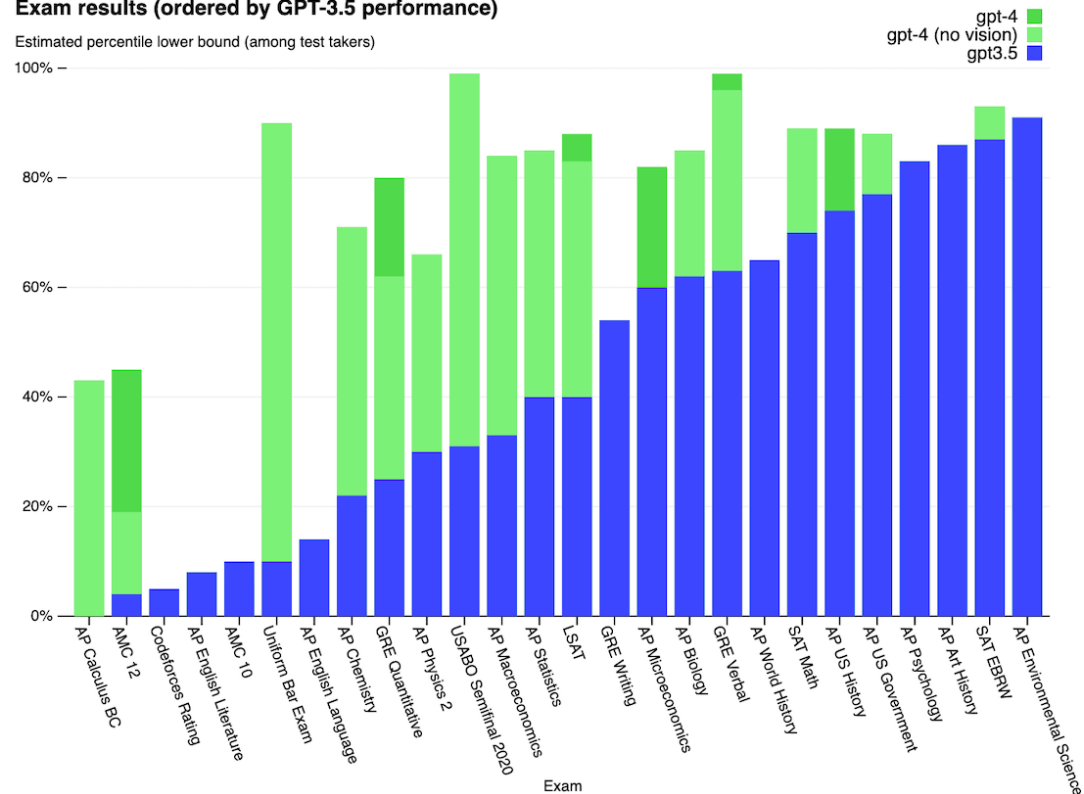
Accuracy



On nine categories of internal adversarially-designed factual evals, we compare GPT-4 (green) to the first three ChatGPT versions. There are significant gains across all topics. An accuracy of 1.0 means the model's answers are judged to be in agreement with human ideal responses for all questions in the eval.

Exam results (ordered by GPT-3.5 performance)

Estimated percentile lower bound (among test takers)

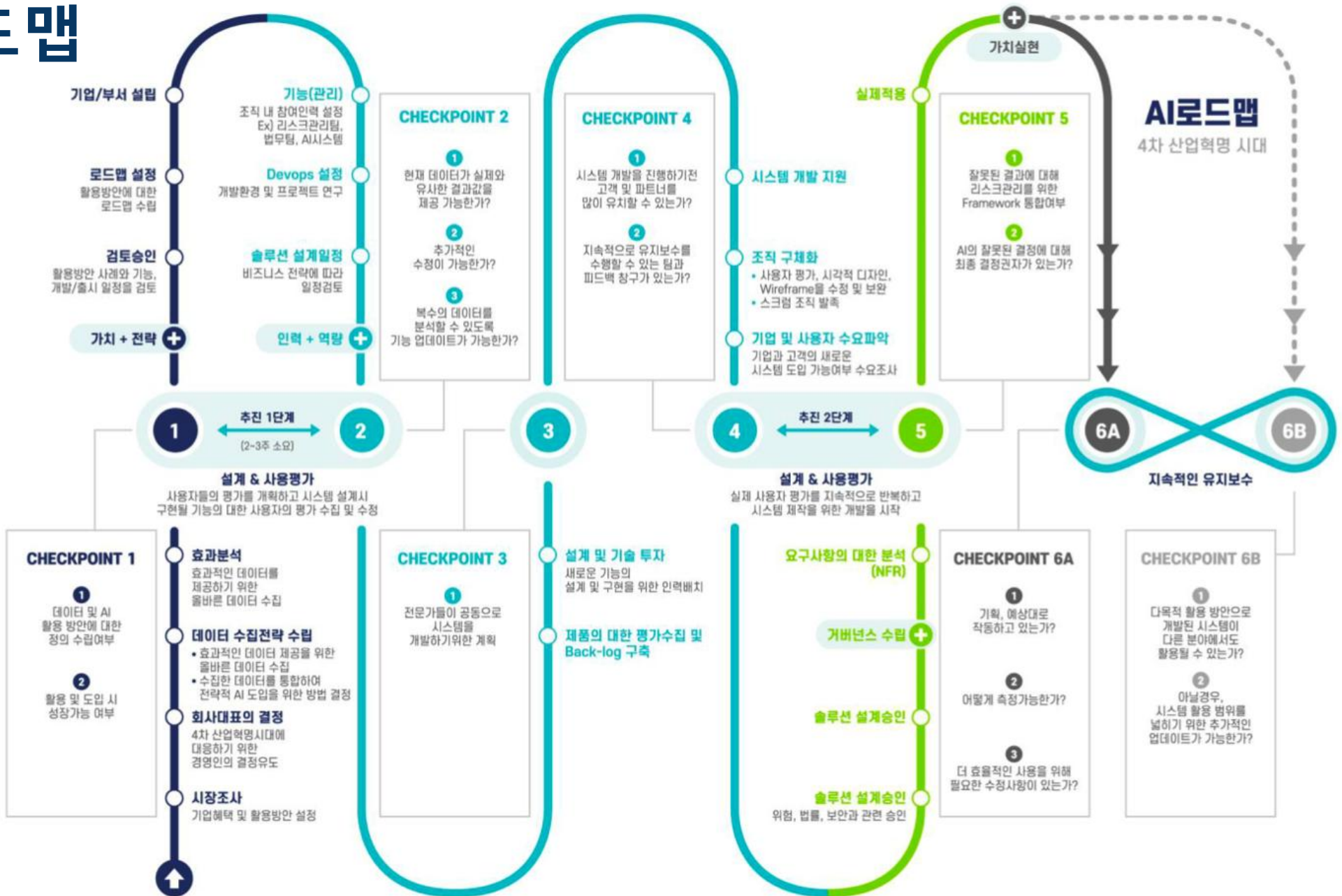


AI Project Pipeline 및 추천 Tool

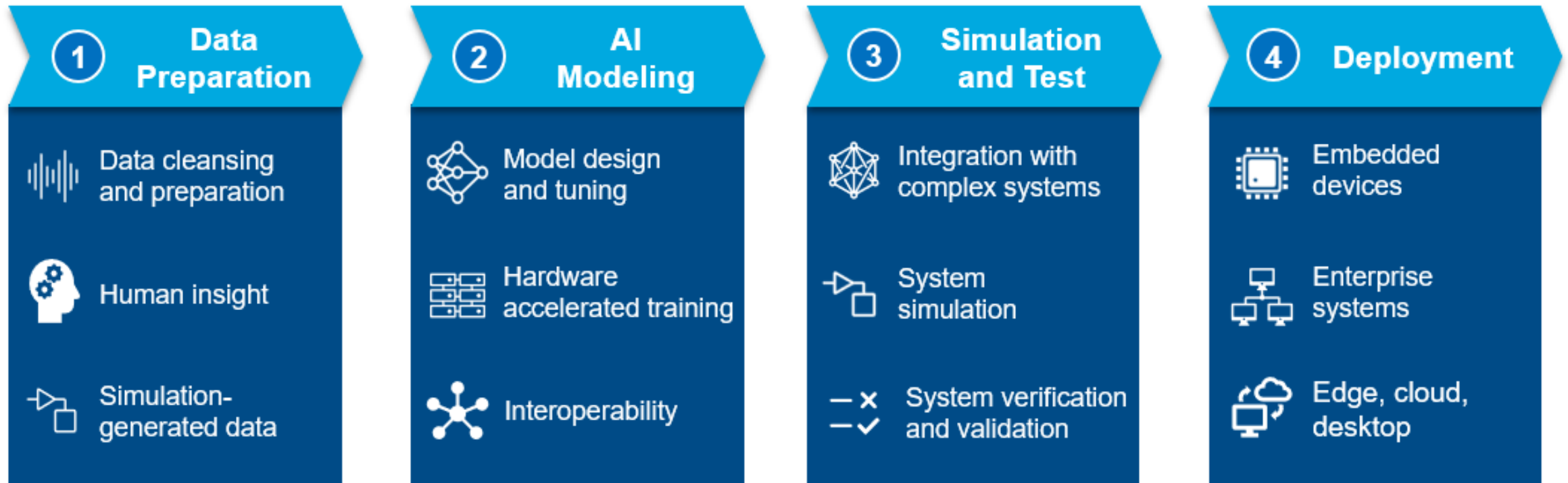
AI Project Pipeline 및 추천 Tool

- AI Project의 보편적인 Pipeline 구성
- Pipeline 구축 시, 활용하기 좋은 Tool 추천
- Tesla AI Day를 통해 살펴보는 AI 기술력의 세부 구성

시로드맵



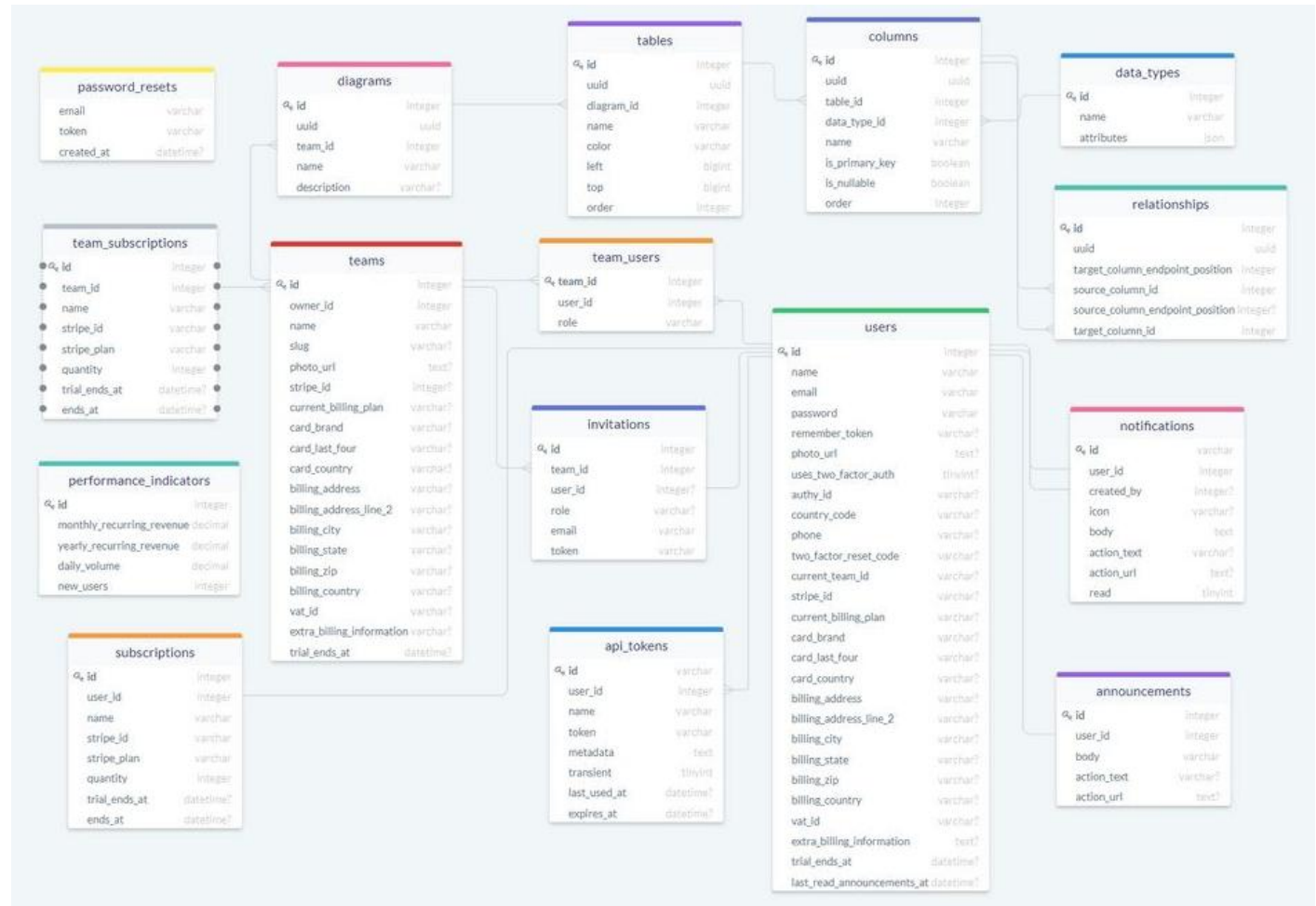
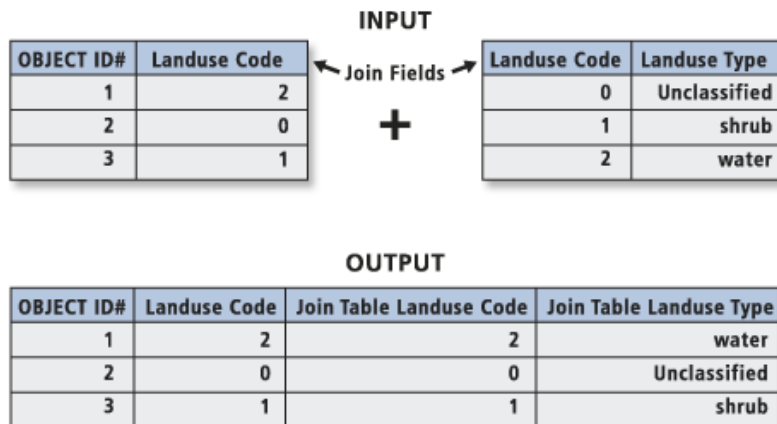
AI Driven Workflow



Detail AI Driven Flow

Data – Modeling & Prediction – Deployment

1. DB Table들에 접근 후
학습에 의미있는 Table들 확인
2. 새로운 학습 Table 생성을 위한
Table Join 진행



Detail AI Driven Flow

Data – Modeling & Prediction – Deployment

3. EDA + Preprocessing Cycle

1. Cycle

1. EDA를 통해서 학습을 하고자 하는 Y:Target과 X:독립변수와의 관계 확인
 - Feature별 분포에 유의미한 차이가 있는지 확인
 - Target과 Not Target Features간의 1차원 / 2차원 통계분석
 1. 1차원 : Bins Graph기반, Feature + Target 분포 확인
 2. 2차원 : Pivot Table 통해서, 입체적으로 어떤 변수의 조합이 Target에 유의미한 영향을 미치는지 파악 (필수는 아님)
 - 독립변수(Not Target Feature) 간 상관관계(Correlation) 분석
 - Test Set 존재한다면 Train / Test Dataset 분포 비교/대조
2. 데이터 1차적, 2차적인 분석학습에 용이할 것으로 보이는 새로운 파생변수 생성 (X:독립변수들 뿐만 아니라 Y:종속변수인 Target의 가능성이 있는 변수들)
3. 학습에 활용이 어려워 보이는 데이터들은 Cleansing 진행

4. 최종 학습 데이터 생성

Detail AI Driven Flow

Data – **Modeling** & Prediction – Deployment

5. Feature Selection

- 학습에 활용할 독립변수(Not Target)/종속변수(Target) 선택

6. Data Splitting

- Train / Valid / Test Dataset
- Train / Valid의 경우 Modeling dataset으로 묶일 수 있으며, K-fold, Bootstrap 기반 학습을 진행할 수 있음

7. Feature Extraction (변수 추출 / 변수 차원 축소)

8. Algorithm Selection

- ML(Machine Learning) / DL(Deep Learning) 학습
- 지도 / 비지도 / 준지도 학습
- Classifier (Binary | Multiclass | Multilabel) / Regressor / Recommendation / Clustering / Reinforcement Learning / speech recognition / image recognition / Computer Vision 등
- Hyper Parameter Tuning

Detail AI Driven Flow

Data – **Modeling** & Prediction – Deployment

9. Model Summary

- 학습이 끝난 후 Train / Valid / Test Dataset의 예측 결과 별 통계 지표 계산
- Visualization
 - 분위 / 등급별 분석 (Binary Classifier / Regressor)
 - 알고리즘 별 Plotting 방법론 차별화 (Classifier / Regressor / Clustering 등)
 - 알고리즘별 변수 설명력 표현

Detail AI Driven Flow

Data – Modeling & **Prediction** – Deployment

10. Batch Prediction / Realtime Prediction 진행

11. Test dataset 통계 기반 분석

12. Blackbox였던 고도화된 알고리즘을 Explainable AI기법으로 설명 능력 강화

(Feature Spike / 등급별 통계적 차이 기반 / SHAP / LIME 등)

Detail AI Driven Flow

Data – Modeling & Prediction – **Deployment**

- 13. 학습된 Model 실제 현업 서비스에 활용할 수 있도록 Deploy 진행
- 14. 현업에서 쉽게 활용할 수 있도록 Monitoring Service 지원
- 15. 주기에 따른 재학습 환경 구축

AI 기능별 추천 Tool

Data – Modeling & Prediction – Deployment

1. 데이터 수집 파이프라인

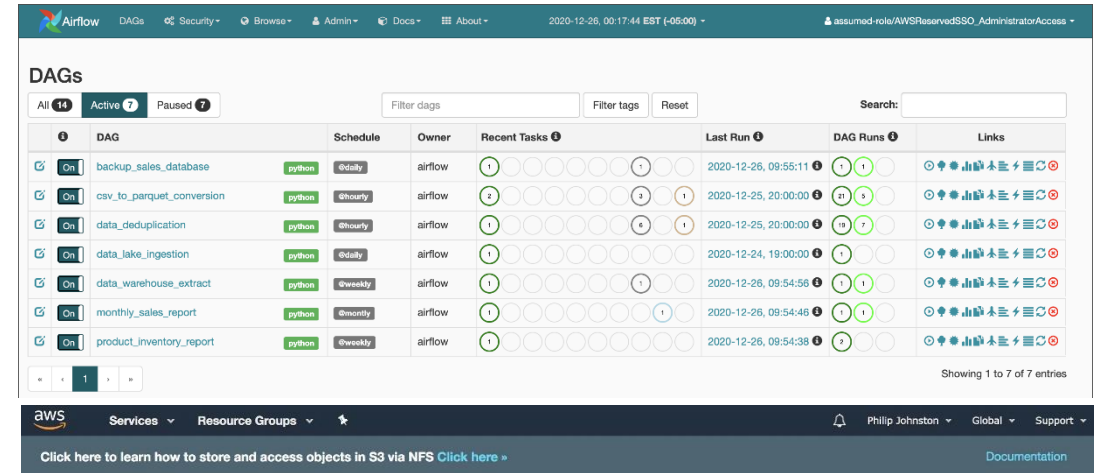
- Sqoop, Flume, Kafka, Flink, Spark Streaming, **Airflow**

2. 데이터 저장

- MySQL, Hadoop, Amazon S3, MinIO

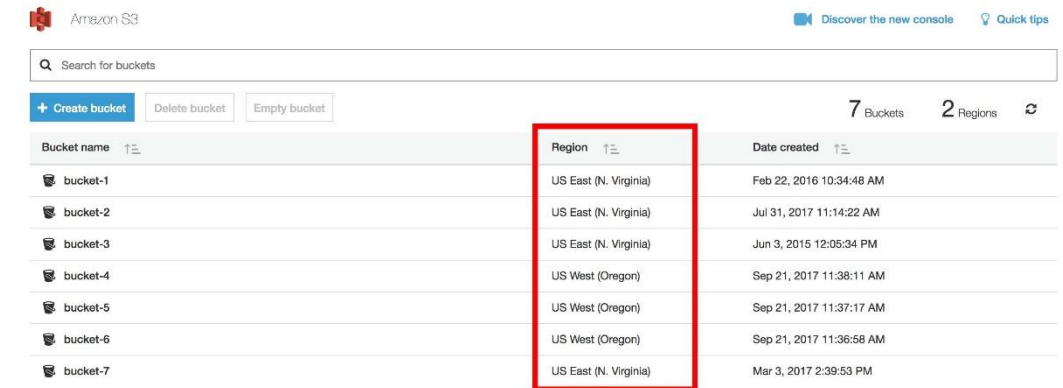
3. 데이터 관리

- TFDV, **DVC**, Feast, Amundsen



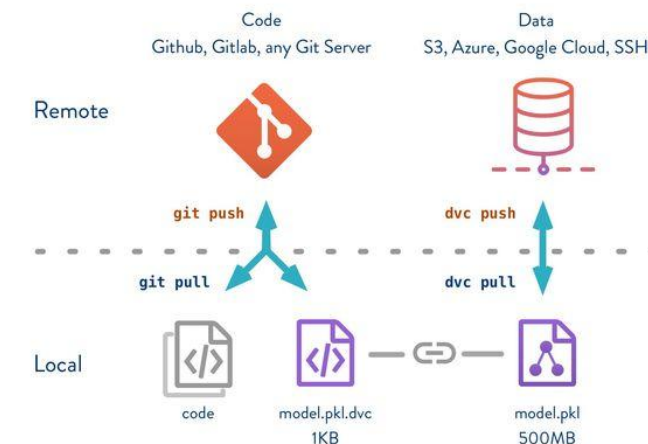
The screenshot shows the Apache Airflow web interface. At the top, there's a navigation bar with links for DAGs, Security, Browse, Admin, Docs, and About. The main header displays the date and time: 2020-12-26, 00:17:44 EST (-06:00). Below the header, there's a section for DAGs with filters for All (14), Active (7), and Paused (7). A table lists several DAGs, including backup_sales_database, csv_to_parquet_conversion, data_deduplication, data_lake_ingestion, data_warehouse_extract, monthly_sales_report, and product_inventory_report. Each row shows the DAG name, schedule, owner, recent tasks, last run, and DAG runs. The bottom of the interface shows the AWS logo and a link to learn how to store and access objects in S3 via NFS.

DAG	Schedule	Owner	Recent Tasks	Last Run	DAG Runs
backup_sales_database	python @daily	airflow	1	2020-12-26, 09:55:11	1
csv_to_parquet_conversion	python @hourly	airflow	2	2020-12-25, 20:00:00	2
data_deduplication	python @hourly	airflow	1	2020-12-25, 20:00:00	1
data_lake_ingestion	python @daily	airflow	1	2020-12-24, 19:00:00	1
data_warehouse_extract	python @weekly	airflow	1	2020-12-26, 09:54:56	1
monthly_sales_report	python @monthly	airflow	1	2020-12-26, 09:54:46	1
product_inventory_report	python @weekly	airflow	1	2020-12-26, 09:54:38	1



The screenshot shows the Amazon S3 console. At the top, there's a search bar for buckets. Below it, there's a table listing buckets. The table has columns for Bucket name, Region, and Date created. The Region column is highlighted with a red box. The buckets listed are bucket-1 through bucket-7, all in the US East (N. Virginia) region.

Bucket name	Region	Date created
bucket-1	US East (N. Virginia)	Feb 22, 2016 10:34:48 AM
bucket-2	US East (N. Virginia)	Jul 31, 2017 11:14:22 AM
bucket-3	US East (N. Virginia)	Jun 3, 2015 12:05:34 PM
bucket-4	US West (Oregon)	Sep 21, 2017 11:38:11 AM
bucket-5	US West (Oregon)	Sep 21, 2017 11:37:17 AM
bucket-6	US West (Oregon)	Sep 21, 2017 11:36:58 AM
bucket-7	US East (N. Virginia)	Mar 3, 2017 2:39:53 PM



AI 기능별 추천 Tool

Data – Modeling & Prediction – Deployment

4. 모델 개발

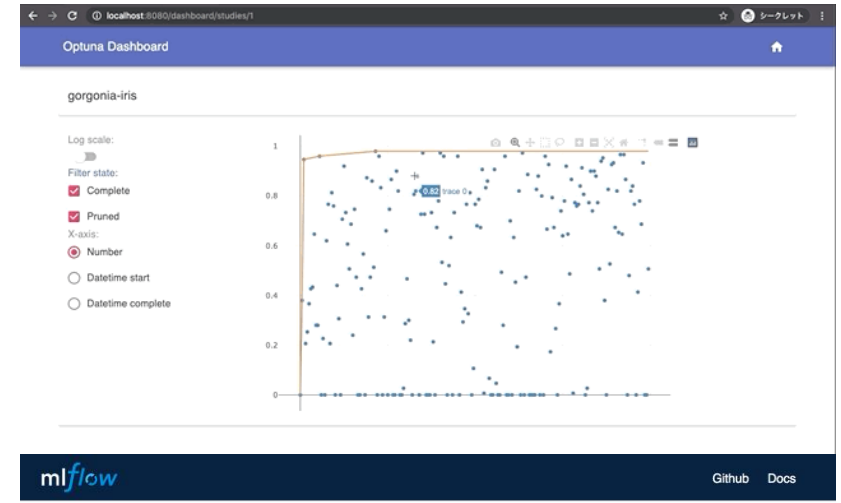
- Jupyter Hub, **Docker**, **Kubeflow**, **Optuna**, **Ray**, **Katib**

5. 모델 버전 관리

- Git, **MLflow**, **Github Action**, **Jenkins**

6. 모델 학습 및 스케줄링 관리

- **Grafana**, **Kubernetes**

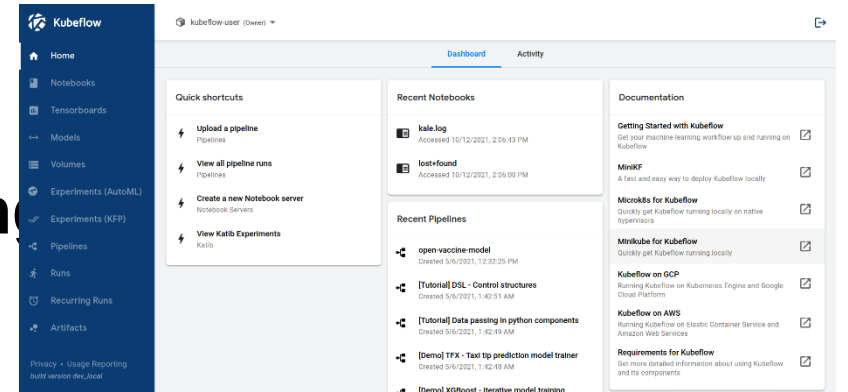


AI 기능별 추천 Tool

Data – Modeling & Prediction – Deployment(Serving)

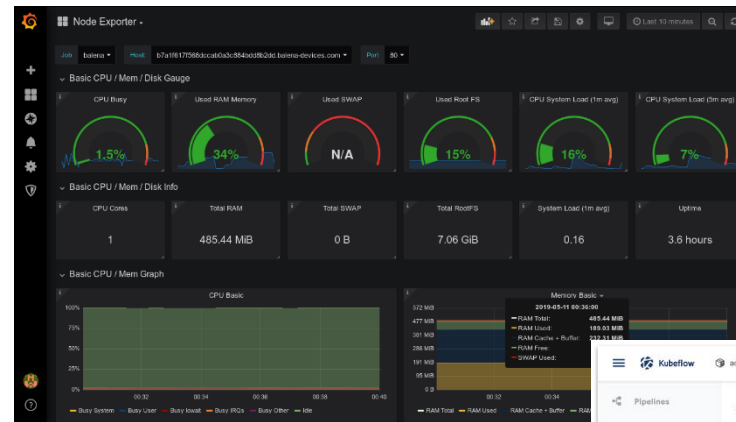
7. 모델 패키징

- **Docker**, Flask, **FastAPI**, **Mlflow**, BentoML, Kubeflow, TFServing, Seldon-Core



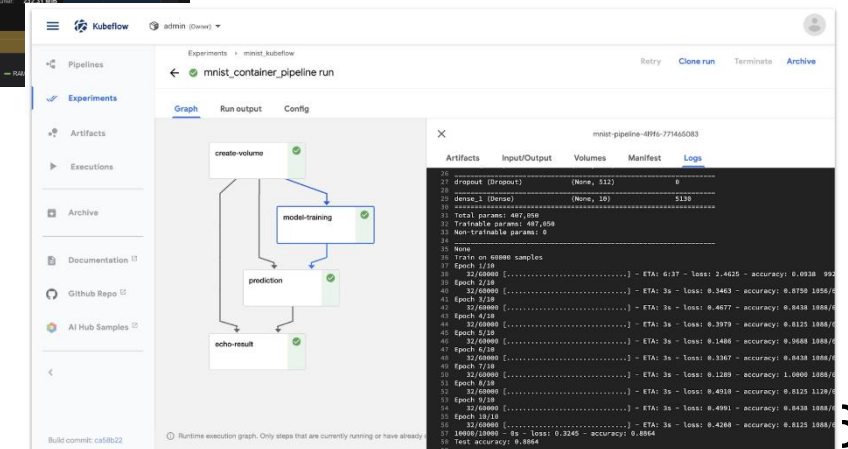
8. 서빙 모니터링

- **Prometheus**, **Grafana**, Thanos



9. 파이프라인 관리

- Kubeflow, Argo Workflows, **Airflow**



Tesla의 AI 활용법 (Tesla's AI Day)

테슬라 데이터 관리 및 컴퓨터 비전 for Autonomous Driving

테슬라가 데이터를 다루는 방법

테슬라 CVPR 2021 분석 - 컴퓨터비전시스템 / 지도학습 / 자동-라벨링 데이터셋 / 슈퍼컴퓨터 / 그림자모드 / 트랜스포머

테슬라 AI 데이 정리 PART1 - 자율주행 / 트랜스포머 / 예측 / 캘리브레이션 / 프로젝트 / 멀티카메라 네트워크 / 비디오모듈 / Spatial RNN

테슬라 AI 데이 정리 PART2 - 자율주행 / Planning / 벡터공간 / 라벨링방법/ 오토 라벨링 파이프라인 / 3D 포인트 클라우드 / 시뮬레이션

DevOps -> MLOps -> LLMOps

DevOps -> MLOps -> LLMOps

- DevOps(Development & Operations)란?
- MLOps(Machine Learning & Ops)란?
- LLMOps(Large Language Model & Ops)란?
- ChatGPT(LLM 서비스) 활용 가이드 및 추천 역량

DevOps(Development & Operations)

DevOps란?

- "Development"와 "Operations"의 합성어
- 소프트웨어 개발(Development)과 IT 운영(Operations) 사이의 문화, 기법, 도구 및 조직 변경을 포괄하는 용어

DevOps의 주요 목표

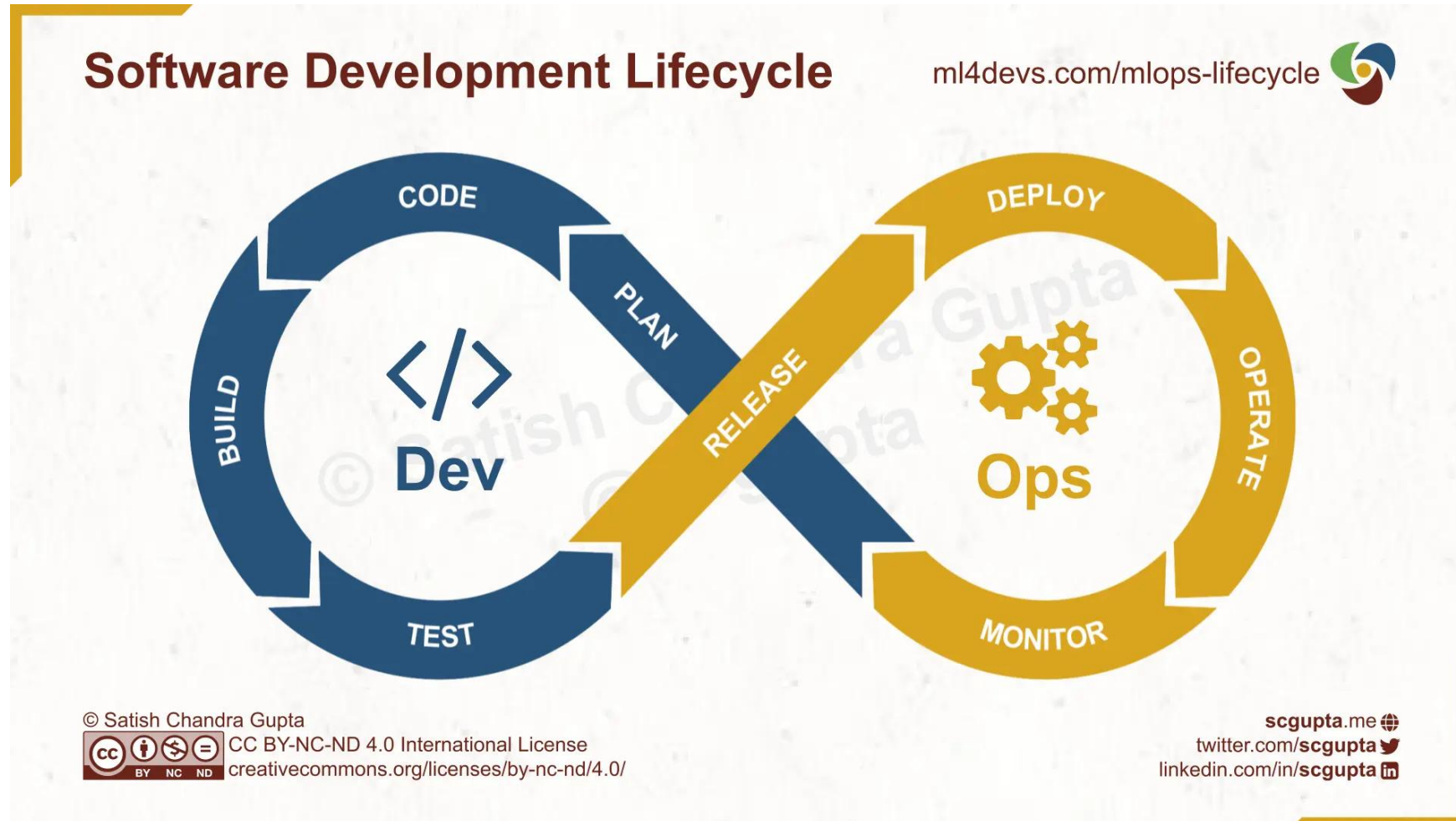
- 개발 및 IT 운영 팀 간의 협력을 증진함으로써 보다 빠르고 안정적으로 고품질의 소프트웨어 제품 및 서비스를 제공하는 것

DevOps의 주요 특징 및 원칙

- 문화적 변화: 개발과 운영 팀 간의 상호 작용과 협력을 증진하며, 서로의 작업을 이해하고 존중하는 문화를 구축합니다.
- 자동화: 배포, 테스트, 모니터링 등의 프로세스를 자동화하여 소프트웨어 제품의 품질을 향상시키고 빠르게 제공할 수 있도록 합니다.
- 지속적 통합 및 지속적 배포 (CI/CD): 코드 변경이 자주 통합되어 항상 배포 가능한 상태를 유지합니다.
- 피드백 루프: 운영 환경에서의 성능 정보와 이슈를 빠르게 개발팀에 전달하여 지속적으로 개선할 수 있게 합니다.
- 인프라스트럭처 자동화 및 코드화: 인프라스트럭처를 코드로 관리(IaC: Infrastructure as Code)하여 환경 구축의 일관성 및 재사용성을 보장합니다.
- 레지리언시(Resiliency): 시스템과 애플리케이션이 장애에 강하도록 설계하며, 장애 발생 시 빠르게 복구할 수 있도록 합니다.

DevOps는 단순히 도구나 기술만을 의미하는 것이 아닌, 조직 내 협업과 효율성을 향상시키기 위한 문화와 접근 방식의 변화를 포함
이는 비즈니스 요구사항에 빠르게 대응하며, 높은 품질의 소프트웨어 제품 및 서비스를 지속적으로 제공하는 중요한 역할을 함

DevOps(Development & Operations)



DevOps 대표 Tool

버전 관리 (Version Control)

- **Git**: 분산 버전 관리 시스템으로 가장 널리 사용
- **GitHub, GitLab, Bitbucket**: Git 저장소 호스팅 서비스로, 코드 협업과 코드 리뷰, CI/CD 통합 등의 기능을 제공

지속적 통합 및 지속적 배포 (CI/CD)

- **Jenkins**: 오픈소스 자동화 서버로, CI/CD 파이프라인 구축에 널리 사용
- **CircleCI, Travis CI**: 클라우드 기반 CI/CD 서비스
- **GitLab CI/CD**: GitLab의 CI/CD 기능
- **GitHub Actions**: GitHub의 CI/CD 및 자동화 기능

컨테이너 및 오케스트레이션

- **Docker**: 애플리케이션을 컨테이너화하여 일관된 환경에서 실행
- **Kubernetes**: 컨테이너화된 애플리케이션의 배포, 확장 및 관리를 위한 오케스트레이션 플랫폼

구성 관리 (Configuration Management)

Ansible: 단순한 YAML 구문을 사용하여 자동화 스크립트를 작성하는 도구

Puppet, Chef: 인프라의 구성을 코드로 관리하고 자동화하는 도구

모니터링 및 로깅

Prometheus: 오픈소스 모니터링 및 경고 시스템

Grafana: 시각화 대시보드를 제공하는 도구로, Prometheus와 잘 통합됨

ELK Stack (Elasticsearch, Logstash, Kibana): 로그 수집, 저장, 검색, 분석 및 시각화를 위한 도구 조합

Splunk: 로그 분석 및 시각화 도구

코드 품질 및 검사

SonarQube: 소스 코드 품질을 검사하고 관리하는 도구

MLOps(Machine Learning & Ops)

MLOps란?

- "Machine Learning"과 "Operations"의 합성어
- DevOps의 원칙과 방법론을 기계 학습(Machine Learning)에 적용한 것
- MLOps는 기계 학습 모델의 개발부터 배포, 모니터링에 이르기까지의 전체 라이프사이클을 관리하는 방법론과 프로세스

MLOps의 주요 목표

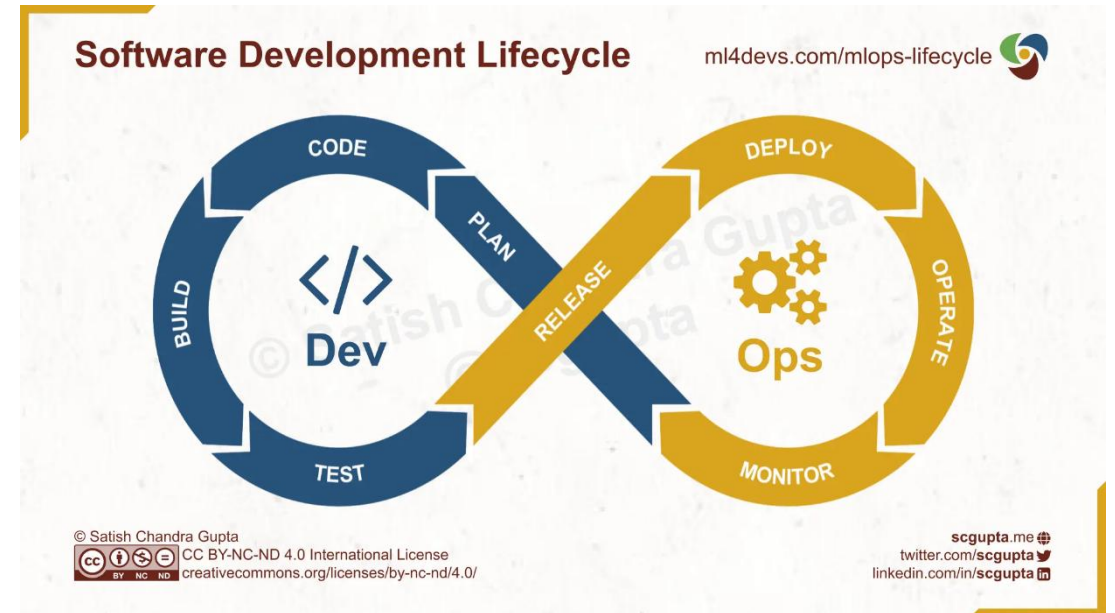
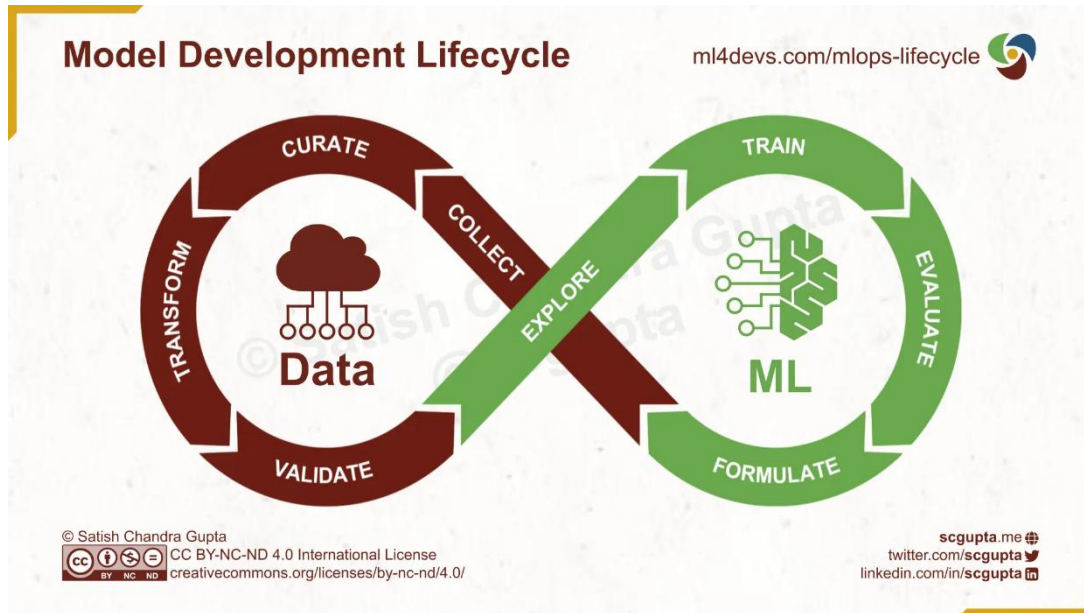
- 기계 학습 모델의 빠른 개발 및 배포
- 모델의 품질 및 성능 감시
- 모델의 안정적이고 지속적인 운영

MLOps의 주요 특징 및 원칙

- 자동화 및 지속적 통합/배포 (CI/CD): 모델 학습, 테스트, 배포 프로세스의 자동화를 통해 모델을 지속적으로 통합 및 배포
- 모니터링 및 로깅: 운영 환경에서의 모델 성능, 입력 데이터의 품질 및 예측값을 실시간으로 모니터링하고 기록
- 재사용성: 전처리, 학습 및 배포 파이프라인을 모듈화하고 재사용 가능하게 만들어, 새로운 모델 또는 기능의 개발 시간을 단축
- 인프라스트럭처 자동화: IaC (Infrastructure as Code) 원칙을 따라 모델 학습과 배포에 필요한 인프라를 자동으로 설정하고 관리
- 피드백 루프: 실제 환경에서의 모델 성능을 지속적으로 모니터링하고, 이를 통한 피드백을 받아 모델을 지속적으로 개선
- 협업 및 문화: 데이터 과학자, 개발자, IT 전문가들이 효과적으로 협업하여, 모델의 개발부터 배포 및 운영까지의 프로세스를 효율적으로 관리

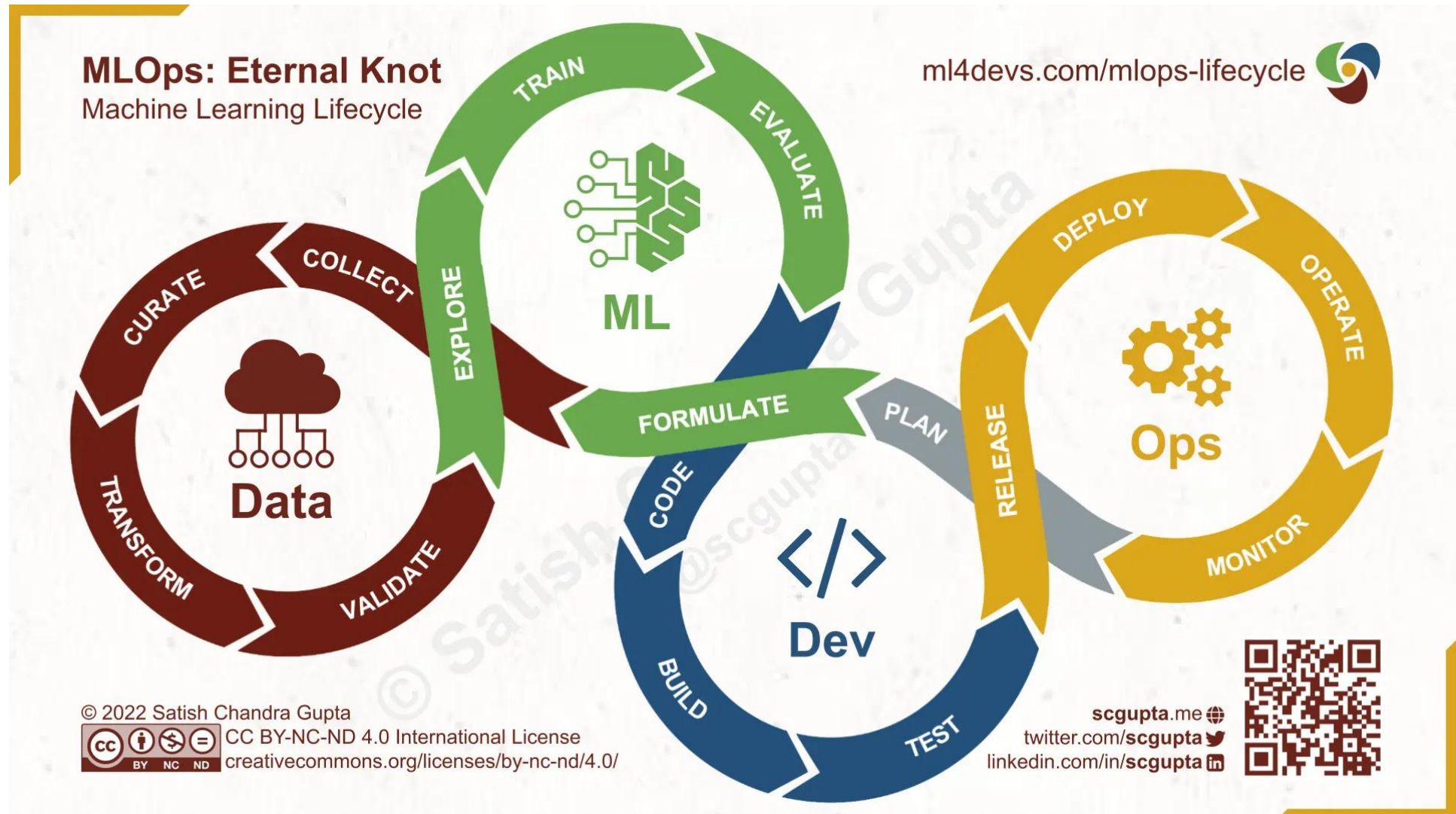
MLOps(Machine Learning & Ops)란?

Data+ML & DevOps 시스템



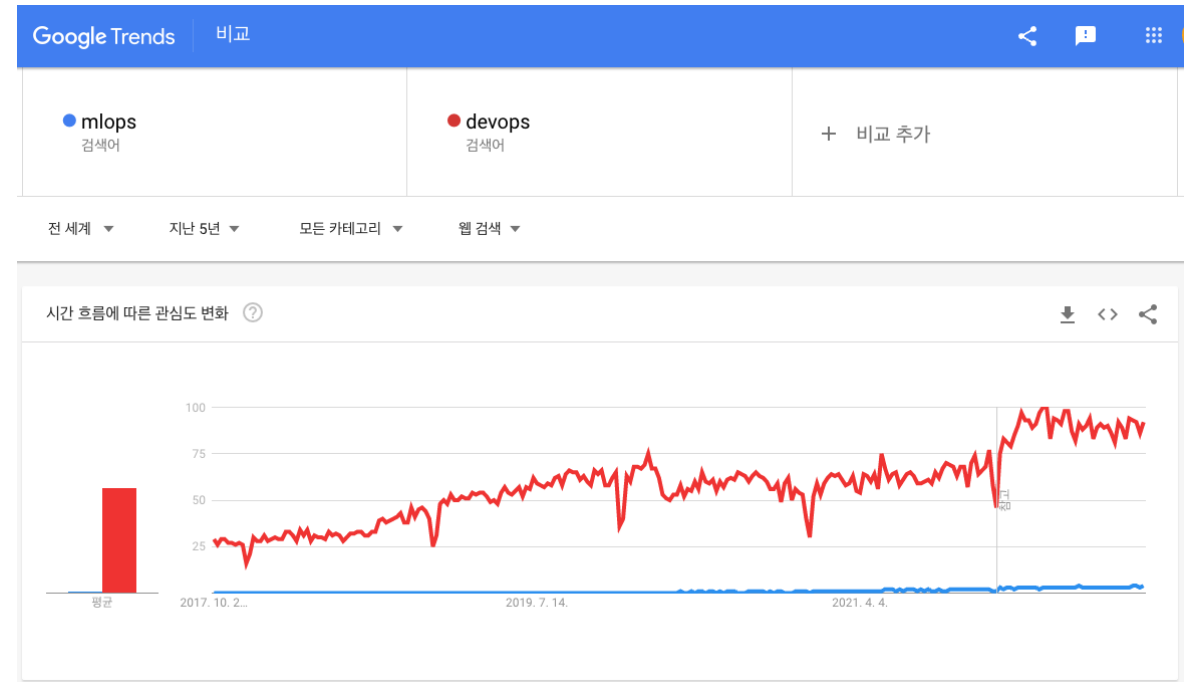
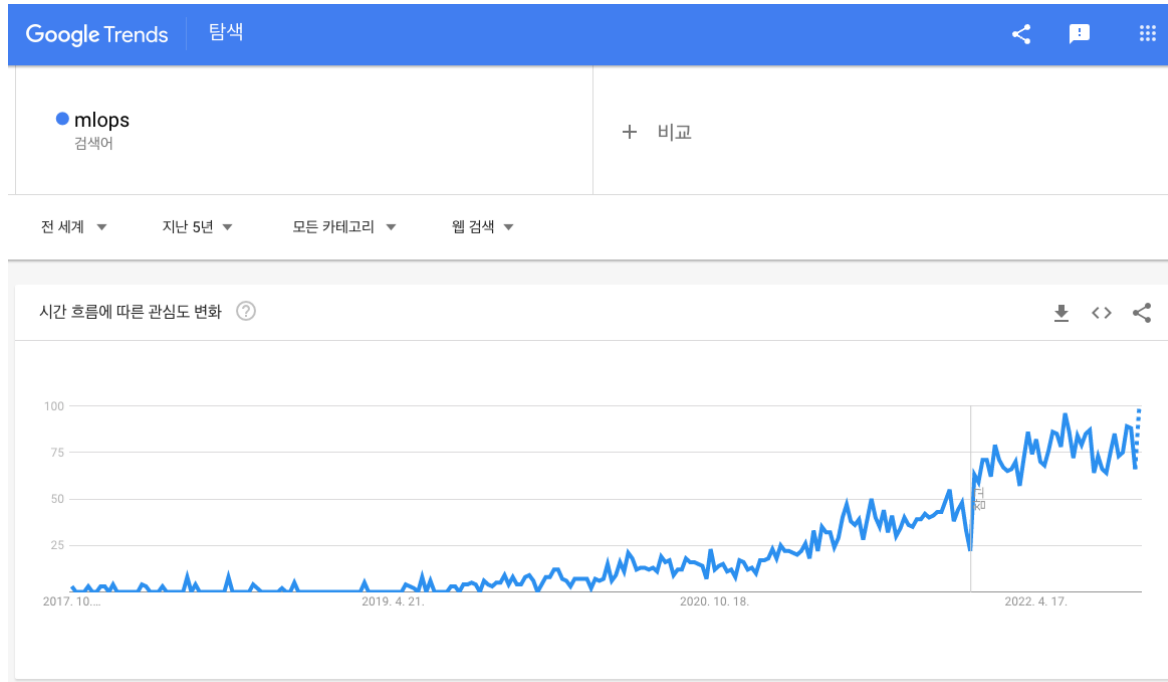
MLOps란?

Data+ML & DevOps 시스템



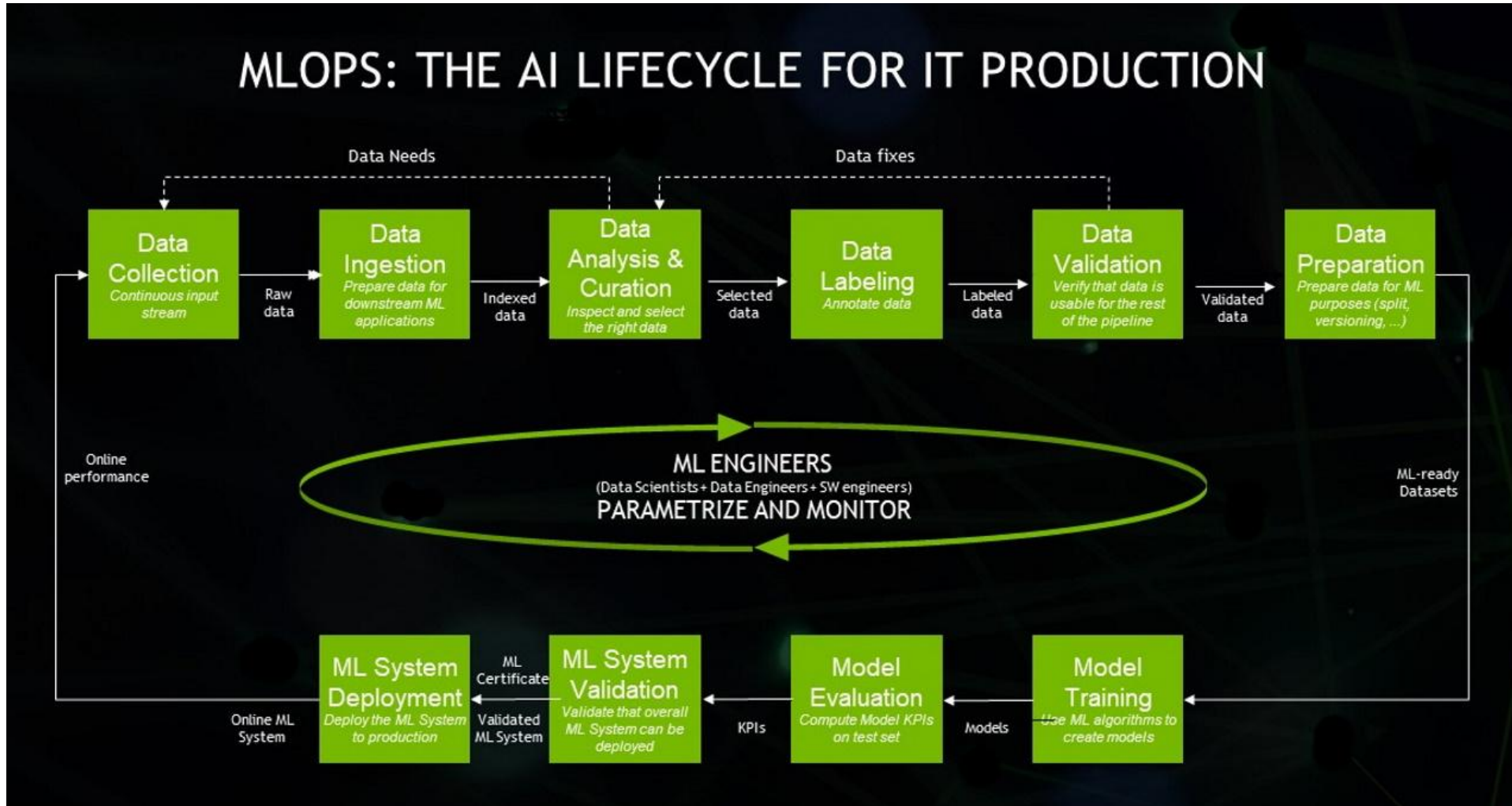
MLOps란?

Trend를 통한 MLOps의 기회



MLOps Architecture

Nvidia[®] | MLOps



MLOps Architecture

Google의 MLOps Level 0

MLOps 수준 0: 수동 프로세스

많은 팀에는 최첨단 모델을 빌드할 수 있는 데이터 과학자와 ML 연구원이 있지만, ML 모델을 빌드하고 배포하는 과정은 완전히 수동으로 이루어집니다. 이는 콘텐츠 성숙도의 기본 수준 또는 수준 0으로 간주됩니다. 다음 다이어그램은 이 프로세스의 워크플로를 보여줍니다.

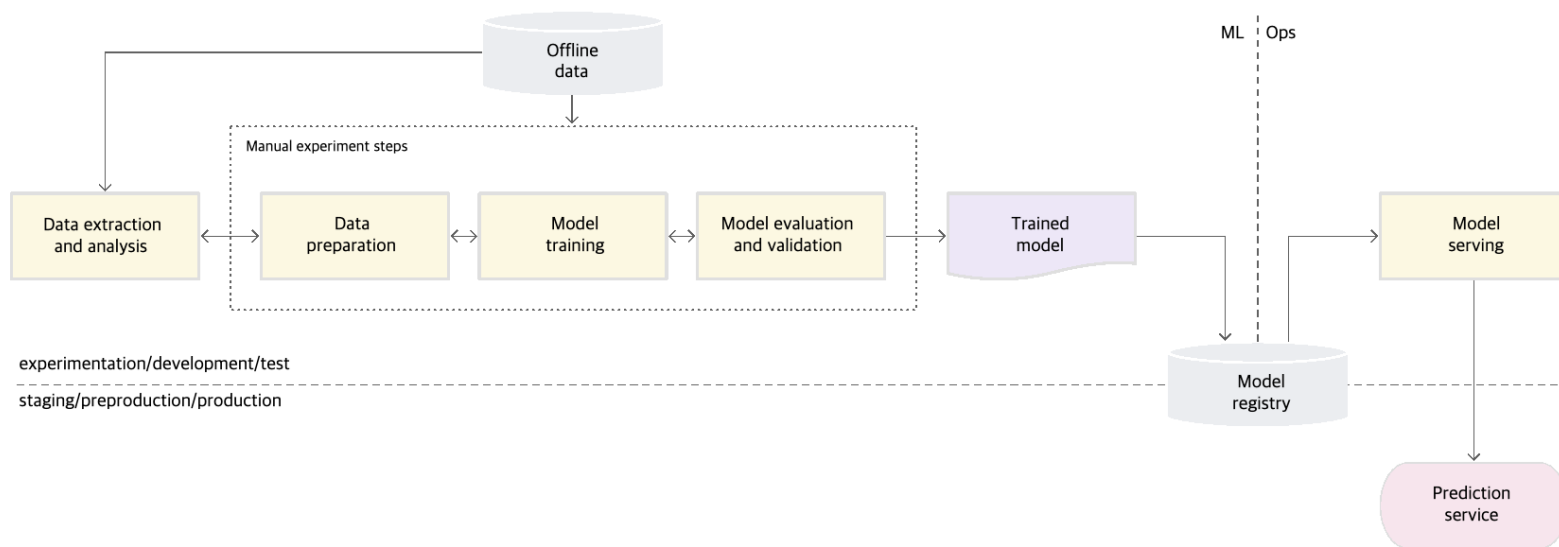


그림 2. 모델을 예측 서비스로 제공하기 위한 수동 ML 단계

MLOps Architecture

Google의 MLOps Level 1

MLOps 수준 1: ML 파이프라인 자동화

수준 1의 목표는 ML 파이프라인을 자동화하여 모델을 지속적으로 학습시키는 것입니다. 이를 통해 모델 예측 서비스를 지속적으로 제공할 수 있습니다. 새 데이터를 사용하여 프로덕션 단계에서 모델을 재학습시키는 프로세스를 자동화하려면 파이프라인 트리거 및 메타데이터 관리뿐만 아니라 자동화된 데이터 및 모델 검증 단계를 파이프라인에 도입해야 합니다.

다음 그림은 CT용 자동화된 ML 파이프라인을 도식화하여 보여줍니다.

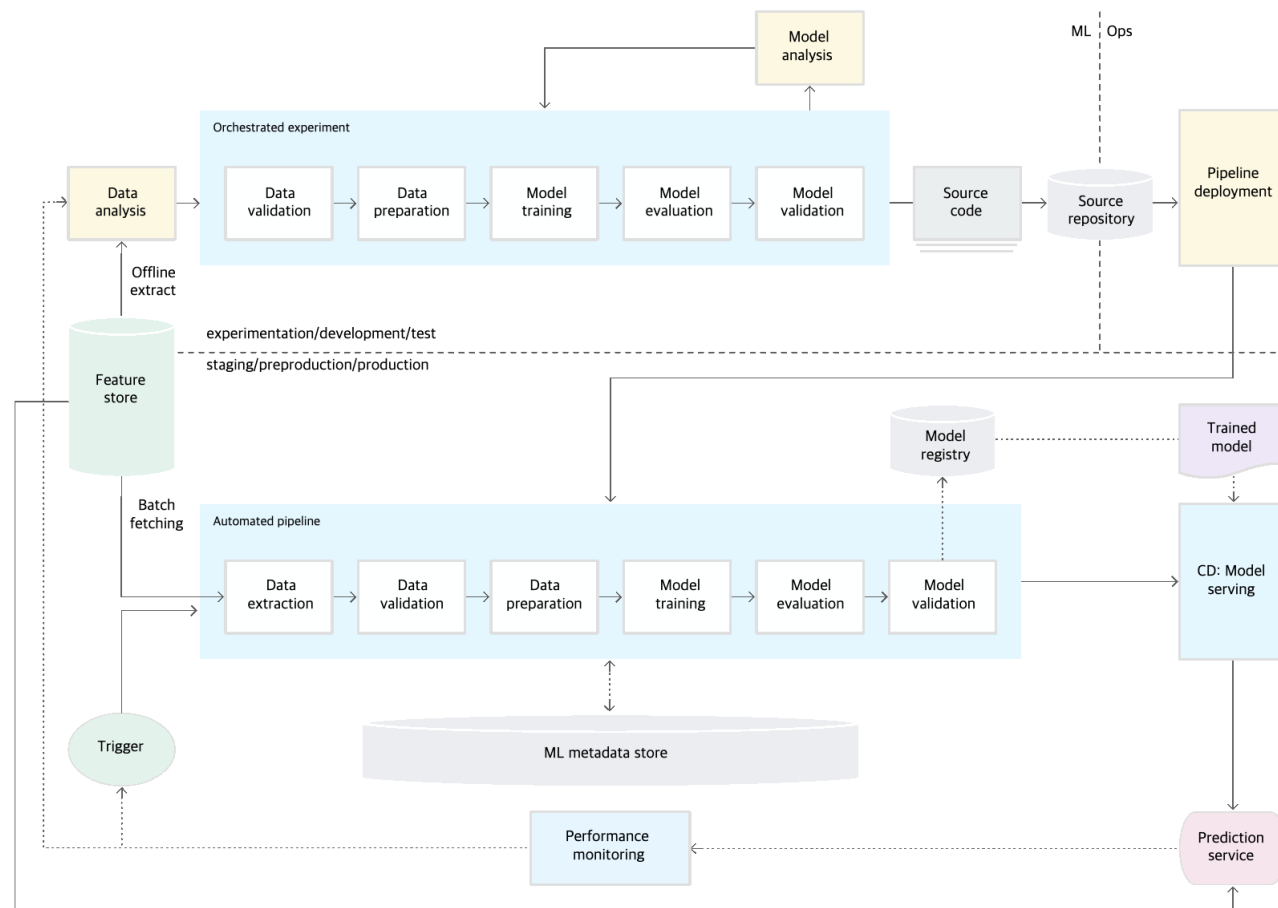


그림 3. CT용 ML 파이프라인 자동화

MLOps Architecture

Google의 MLOps Level 2

MLOps 수준 2: CI/CD 파이프라인 자동화

프로덕션 단계에서 파이프라인을 빠르고 안정적이게 업데이트하려면 강력한 자동화된 CI/CD 시스템이 필요합니다. 이 자동화된 CI/CD 시스템을 사용하면 데이터 과학자가 특성 추출, 모델 아키텍처, 초매개변수에 대한 새로운 아이디어를 빠르게 살펴볼 수 있습니다. 데이터 과학자는 이러한 아이디어를 구현하고 새 파이프라인 구성요소를 대상 환경에 자동으로 빌드, 테스트, 배포할 수 있습니다.

다음 다이어그램은 자동화된 ML 파이프라인 설정과 자동화된 CI/CD 루틴의 특성을 가진 CI/CD를 사용하여 ML 파이프라인을 구현하는 방법을 보여줍니다.

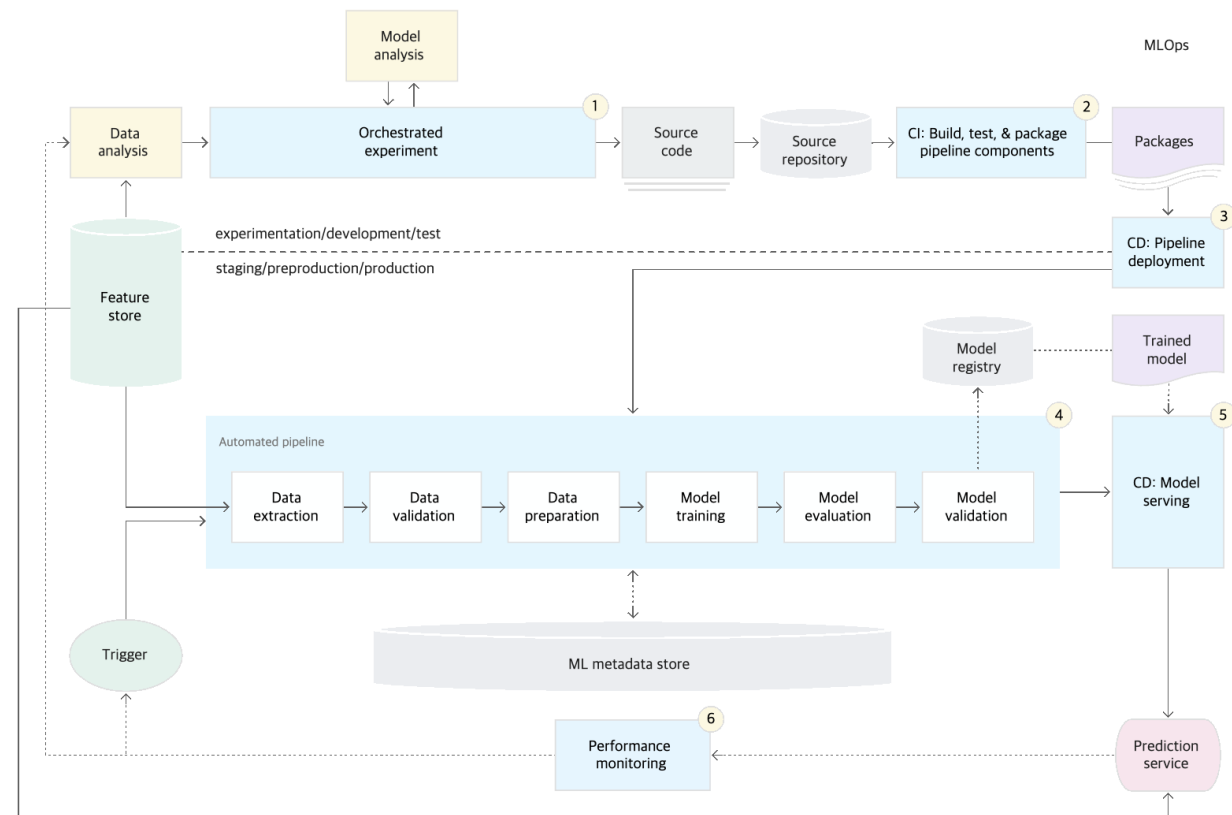
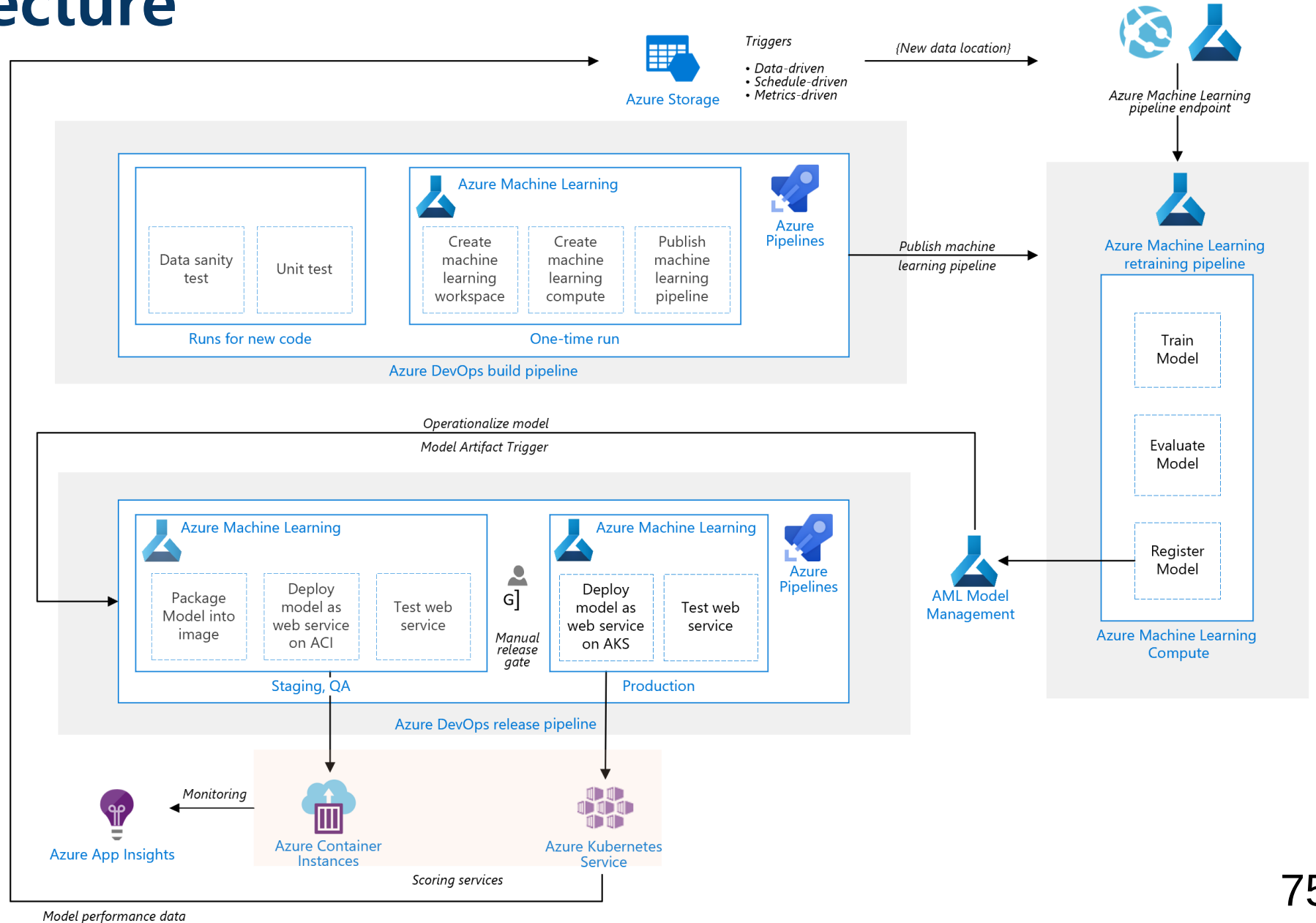


그림 4. CI/CD 및 자동화된 ML 파이프라인

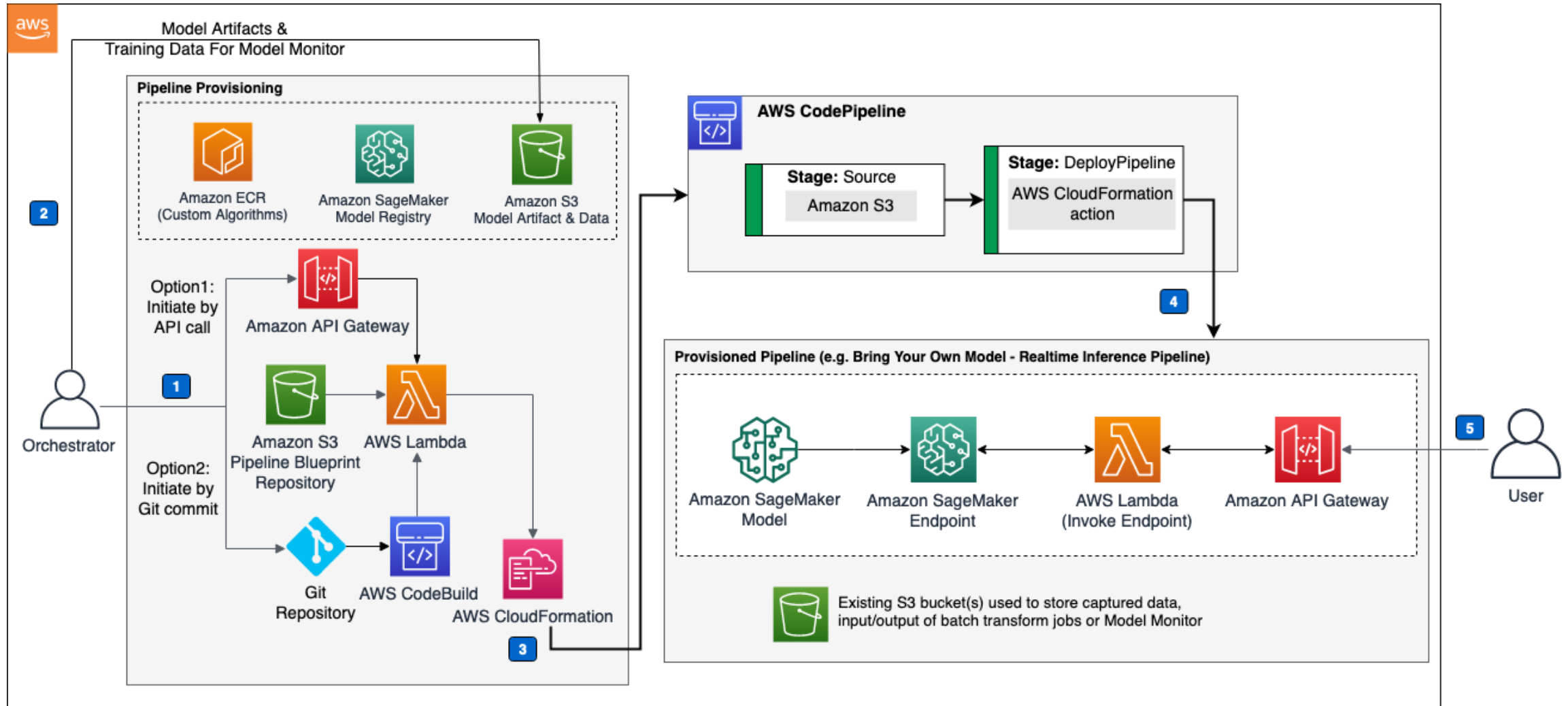
MLOps Architecture

Microsoft의 MLOps



MLOps Architecture

Amazon MLOps



LLMOps(Large Language Model & Ops)

- LLM이란?
 - "Large Language Model"은 큰 규모의 데이터셋에서 훈련된 딥 러닝 기반의 언어 모델
 - OpenAI의 GPT-2, GPT-3, 그리고 이번 대화에서 사용되는 GPT-4는 대표적인 LLM의 예시
 - 인터넷의 글, 책, 기사 등 다양한 텍스트 데이터를 통해 사람처럼 텍스트를 이해하고 생성할 수 있음
 - 다양한 언어 작업들에 적용이 가능하며, 텍스트 생성, 질의 응답, 번역 등 다양한 분야에서 활용
- LLMOps란?
 - **대규모 언어 모델에 특화된 운영 작업**
(즉, 기존 고사양 PC 혹은 서버로는 감당하기 어려운 정도의 대용량 모델을 활용하게 되므로, Cloud 내에서 작동 가능성 및 서비스 확장 가능성을 고려하여 MLOps 환경을 구현하는 것!)

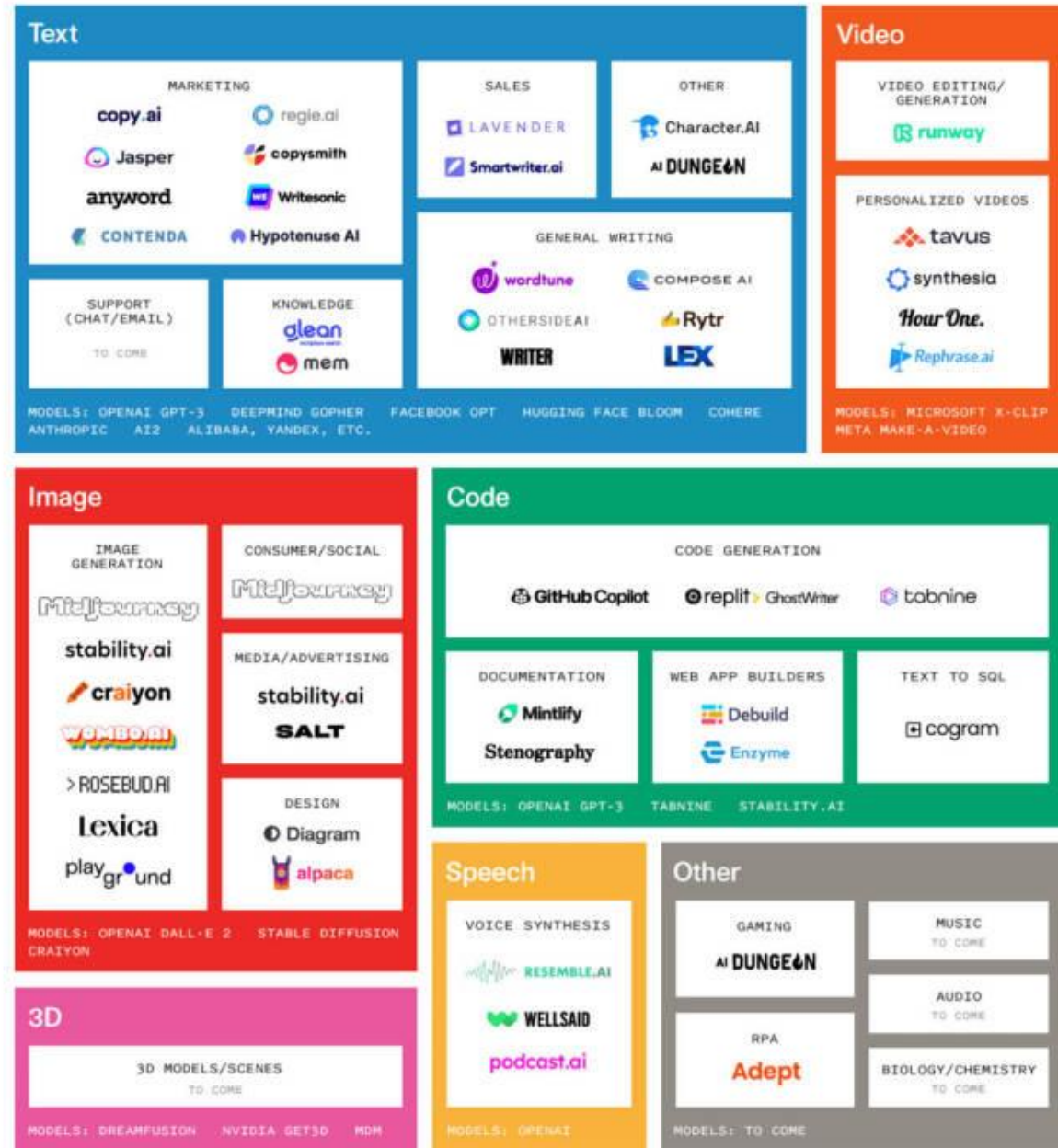
LLMOps

- Pre-training (사전 학습)
 - 대량의 텍스트 데이터를 사용해 모델을 처음부터 학습 (이 때 모델은 문장의 다음 단어나 문장을 예측하도록 학습)
 - 이 과정을 통해 모델은 문법, 문맥, 세계 지식 등을 습득
- Fine-tuning (파인 튜닝)
 - 사전 학습된 모델을 특정 작업에 맞게 추가로 학습시키는 과정
 - 예를 들어, 특정 주제나 도메인의 데이터를 사용하여 모델을 학습시켜 그 주제나 도메인에 특화된 응답을 할 수 있게 만들 수 있음
- Prompt tuning (프롬프트 튜닝)
 - 모델에게 특정 방식으로 입력을 제공하여 원하는 출력을 얻는 방법
 - 사전 학습이나 정교한 튜닝 없이도 특정 작업에 대해 좋은 성능을 내도록 가이드하는데 사용됩니다.
- Few-shot learning (퓨샷 학습)
 - 모델에게 소량의 예시 데이터를 제공하여 특정 작업을 수행하도록 학습시키는 방법입니다.
 - 모델이 이전에 학습한 지식을 바탕으로 새로운 작업을 빠르게 습득하게 됩니다.
- Zero-shot learning (제로샷 학습)
 - 모델에게 추가적인 학습 예시 없이 바로 특정 작업을 수행하도록 요청하는 방법입니다.
 - 이 방법은 모델이 사전 학습 과정에서 습득한 지식만을 바탕으로 작업을 수행하게 됩니다.
- Continual learning (지속적 학습)
 - 모델이 새로운 데이터나 작업에 대한 정보를 계속 학습하면서 기존에 학습한 내용을 잊지 않도록 하는 방법입니다

LLMOps

생성AI 애플리케이션 지도

- Multimodal AI
 - Text to Image/Video
 - Text to Speech(TTS)
 - Speech to Text(STT)
 - Code Generator



[자료=Sequoia Capital] [그래픽=홍종현 미술가자]

국내 초거대 AI 동향

기업명	주요특징
카카오	● 현재 GPT-3 기반 언어모델 'KoGPT'와 이미지 생성 AI 'Karlo' 보유 ● 상반기 내 한국어 특화 ChatGPT 출시 계획
네이버	● 현재 초거대 AI '하이퍼클로바(HyperCLOVA)' API 형태로 서비스중 ● 상반기 검색 특화 '서치GPT' 출시 및 7월중 초거대AI '하이퍼클로바X' 공개 예정
SKT	● 초거대 대화형 AI 서비스 '에이닷(A.)' 개발 ● 올해 정식 서비스 출시 예정
LG	● 초거대 AI '엑사원(EXAONE)' 보유 ● LG 계열사와 협력 통해 산업 현장에 엑사원적용, LG유플러스 AI 컨택센터 사업 추진중
KT	● 초거대 AI '믿음(MIDEUM)' 보유 ● 금융사 대상 AI 컨택센터 구축 논의

[그래픽=홍종현 미술기자]

LLM(Ex. ChatGPT) 서비스 활용 가이드 및 추천 역량

LLM 수준별 활용 정도

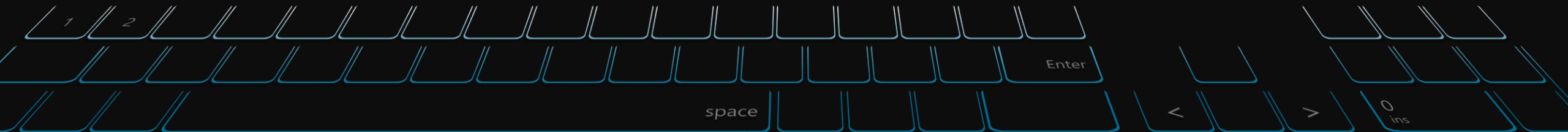
- 초급: GPT3.5, GPT4.0을 OpenAI 화면 혹은 앱에서 활용
- 중급: API로 <GPT3.5-turbo> 기반 <Chat Completion> 기능으로 대화 나누면서 연구/개발 역량 키우기
- 고급: Cloud 기반, GPT처럼 유명한 Foundation Model 기반 직접 활용 & 모델링 & 새로운 소프트웨어에 적용

LLM 관련 추천 역량 및 기술 경험해보기

- DevOps: System 자원 관리(Docker, Kubernetes, Grafana, Prometheus), CI-CD (Github Action, Jenkins) 등
- MLOps: 데이터 버전 관리(DVC), DataOps/MLOps(Airflow X Mlflow), ModelTuning(Optuna, Ray) 등
- LLMOps: 최신 LLM Foundation Model, 최신 Multimodal AI 기술, RAG, CoT 등
- Cloud: AWS, Azure, GCP에서 제공되는 위 DevOps, MLOps, LLMOps 관련 기능들
 - AutoML Tool: AWS SageMaker, Azure ML Studio, GCP Vertex AI
 - AutoML 외 엄청나게 많은 Tool들!!

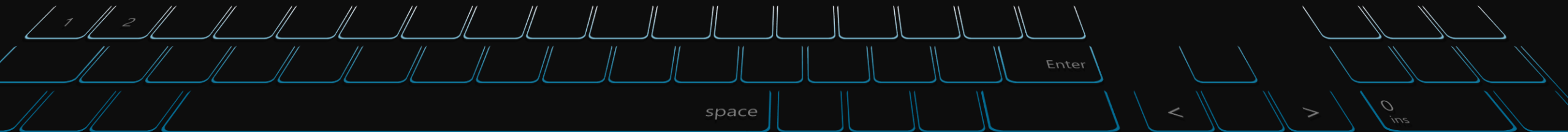
Q&A and Break Time

질의응답 및 휴식시간 (10분)



2부: 회귀와 예측

- 1) 단순 선형 회귀
- 2) 다중 선형 회귀
- 3) 회귀 기반 예측
- 4) Factor 변수





통계학의 가장 일반적인 목표

- "변수 X (또는 $X_1, X_2, \dots, X_n$)는 변수 Y 와 연관되어 있습니까?"로부터
"관계는 무엇이며 Y 를 예측하는 데 사용합니까?"에 대한 근거를 찾아내는 것!
- 통계와 데이터 과학 사이의 연관성은 예측의 영역에서 가장 두드러지게 나타난다.
특히 독립 변수 $X_1, X_2, \dots, X_n$ 들 기반으로 종속(타겟) 변수를 예측한다는 것보다 더 강력한 연관성은 없다

회귀 적용 사례

- 결과가 알려지지 않은 데이터에 대한 후속 적용을 위해 결과가 알려진 데이터에 대한 모델을 훈련하는 이러한 프로세스를 지도 학습이라고 한다. (Supervised Learning)
- 데이터 과학과 통계 간의 또 다른 중요한 연결은 비정상적 기록을 감지하는 데, 데이터 분석 및 회귀 모델 개선을 위한 회귀 진단을 사용할 수 있는 이상 감지 영역이다. (Anomaly Detection)



Simple Linear Regression

Simple linear regression provides a model of the relationship between the magnitude of one variable and that of a second—for example, as X increases, Y also increases. Or as X increases, Y decreases.¹ Correlation is another way to measure how two variables are related—see the section “[Correlation](#)” on [page 30](#). The difference is that while correlation measures the *strength* of an association between two variables, regression quantifies the *nature* of the relationship.

Key Terms for Simple Linear Regression

Response

The variable we are trying to predict.

Synonyms

dependent variable, Y variable, target, outcome

Independent variable

The variable used to predict the response.

Synonyms

X variable, feature, attribute, predictor

Record

The vector of predictor and outcome values for a specific individual or case.

Synonyms

row, case, instance, example

Intercept

The intercept of the regression line—that is, the predicted value when $X = 0$.

Synonyms

b_0 , β_0

Regression coefficient

The slope of the regression line.

Synonyms

slope, b_1 , β_1 , parameter estimates, weights

Fitted values

The estimates $\hat{Y}_i$ obtained from the regression line.

Synonym

predicted values

Residuals

The difference between the observed values and the fitted values.

Synonym

errors

Least squares

The method of fitting a regression by minimizing the sum of squared residuals.

Synonyms

ordinary least squares, OLS



단순 선형 회귀 > The Regression Equation

Simple Linear Regression

The Regression Equation

Simple linear regression estimates how much Y will change when X changes by a certain amount. With the correlation coefficient, the variables X and Y are interchangeable. With regression, we are trying to predict the Y variable from X using a linear relationship (i.e., a line):

$$Y = b_0 + b_1X$$

We read this as “ Y equals b_1 times X , plus a constant b_0 .” The symbol b_0 is known as the *intercept* (or constant), and the symbol b_1 as the *slope* for X . Both appear in R output as *coefficients*, though in general use the term *coefficient* is often reserved for b_1 . The Y variable is known as the *response* or *dependent* variable since it depends on X . The X variable is known as the *predictor* or *independent* variable. The machine learning community tends to use other terms, calling Y the *target* and X a *feature* vector. Throughout this book, we will use the terms *predictor* and *feature* interchangeably.

Consider the scatterplot in **Figure 4-1** displaying the number of years a worker was exposed to cotton dust (Exposure) versus a measure of lung capacity (PEFR or “peak expiratory flow rate”). How is PEFR related to Exposure? It’s hard to tell based just on the picture.

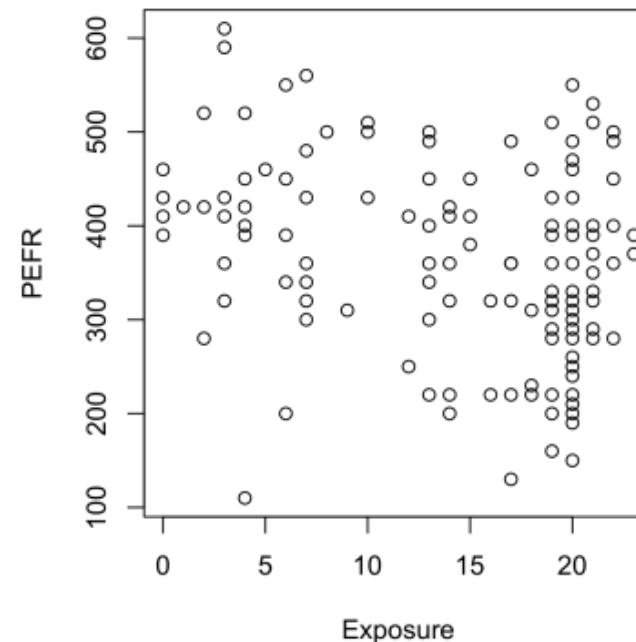


Figure 4-1. Cotton exposure versus lung capacity



단순 선형 회귀 > The Regression Equation

Simple Linear Regression

Simple linear regression tries to find the “best” line to predict the response PEFR as a function of the predictor variable Exposure:

$$\text{PEFR} = b_0 + b_1 \text{Exposure}$$

The `lm` function in R can be used to fit a linear regression:

```
model <- lm(PEFR ~ Exposure, data=lung)
```

`lm` stands for *linear model*, and the `~` symbol denotes that PEFR is predicted by Exposure. With this model definition, the intercept is automatically included and fitted. If you want to exclude the intercept from the model, you need to write the model definition as follows:

```
PEFR ~ Exposure - 1
```

Printing the `model` object produces the following output:

```
Call:
lm(formula = PEFR ~ Exposure, data = lung)
```

```
Coefficients:
(Intercept)  Exposure
  424.583      -4.185
```

The intercept, or b_0 , is 424.583 and can be interpreted as the predicted PEFR for a worker with zero years exposure. The regression coefficient, or b_1 , can be interpreted as follows: for each additional year that a worker is exposed to cotton dust, the worker’s PEFR measurement is reduced by -4.185 .

In *Python*, we can use `LinearRegression` from the `scikit-learn` package. (the `statsmodels` package has a linear regression implementation that is more similar to R (`sm.OLS`); we will use it later in this chapter):

```
predictors = ['Exposure']
outcome = 'PEFR'

model = LinearRegression()
model.fit(lung[predictors], lung[outcome])

print(f'Intercept: {model.intercept_:.3f}')
print(f'Coefficient Exposure: {model.coef_[0]:.3f}')
```

The regression line from this model is displayed in Figure 4-2.

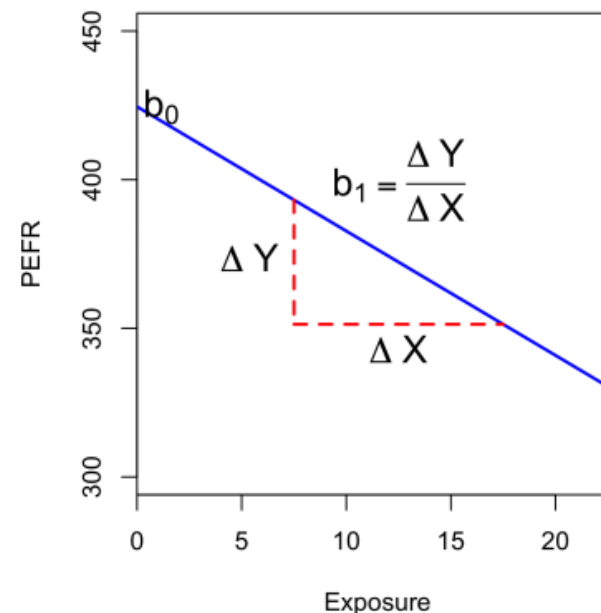


Figure 4-2. Slope and intercept for the regression fit to the lung data



단순 선형 회귀 > Fitted Values and Residuals

Simple Linear Regression

Fitted Values and Residuals

Important concepts in regression analysis are the *fitted values* (the predictions) and *residuals* (prediction errors). In general, the data doesn't fall exactly on a line, so the regression equation should include an explicit error term e_i :

$$Y_i = b_0 + b_1 X_i + e_i$$

The fitted values, also referred to as the *predicted values*, are typically denoted by $\hat{Y}_i$ (Y-hat). These are given by:

$$\hat{Y}_i = \hat{b}_0 + \hat{b}_1 X_i$$

The notation $\hat{b}_0$ and $\hat{b}_1$ indicates that the coefficients are estimated versus known.



Hat Notation: Estimates Versus Known Values

The "hat" notation is used to differentiate between estimates and known values. So the symbol $\hat{b}$ ("b-hat") is an estimate of the unknown parameter b . Why do statisticians differentiate between the estimate and the true value? The estimate has uncertainty, whereas the true value is fixed.²

We compute the residuals $\hat{e}_i$ by subtracting the *predicted* values from the original data:

$$\hat{e}_i = Y_i - \hat{Y}_i$$

In R, we can obtain the fitted values and residuals using the functions `predict` and `residuals`:

```
fitted <- predict(model)
resid <- residuals(model)
```

With `scikit-learn`'s `LinearRegression` model, we use the `predict` method on the training data to get the fitted values and subsequently the residuals. As we will see, this is a general pattern that all models in `scikit-learn` follow:

```
fitted = model.predict(lung[predictors])
residuals = lung[outcome] - fitted
```

Figure 4-3 illustrates the residuals from the regression line fit to the lung data. The residuals are the length of the vertical dashed lines from the data to the line.

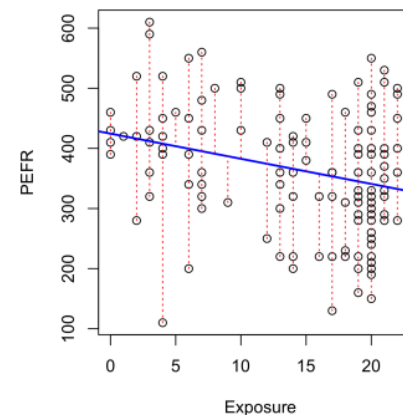


Figure 4-3. Residuals from a regression line (to accommodate all the data, the y-axis scale differs from Figure 4-2, hence the apparently different slope)



단순 선형 회귀 > Least Squares

Simple Linear Regression

Least Squares

How is the model fit to the data? When there is a clear relationship, you could imagine fitting the line by hand. In practice, the regression line is the estimate that minimizes the sum of squared residual values, also called the *residual sum of squares* or *RSS*:

$$\begin{aligned} \text{RSS} &= \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \\ &= \sum_{i=1}^n (Y_i - \hat{b}_0 - \hat{b}_1 X_i)^2 \end{aligned}$$

The estimates $\hat{b}_0$ and $\hat{b}_1$ are the values that minimize RSS.

The method of minimizing the sum of the squared residuals is termed *least squares* regression, or *ordinary least squares* (OLS) regression. It is often attributed to Carl Friedrich Gauss, the German mathematician, but was first published by the French mathematician Adrien-Marie Legendre in 1805. Least squares regression can be computed quickly and easily with any standard statistical software.

Historically, computational convenience is one reason for the widespread use of least squares in regression. With the advent of big data, computational speed is still an important factor. Least squares, like the mean (see [“Median and Robust Estimates” on page 10](#)), are sensitive to outliers, although this tends to be a significant problem only in small or moderate-sized data sets. See [“Outliers” on page 177](#) for a discussion of outliers in regression.



Regression Terminology

When analysts and researchers use the term *regression* by itself, they are typically referring to linear regression; the focus is usually on developing a linear model to explain the relationship between predictor variables and a numeric outcome variable. In its formal statistical sense, regression also includes nonlinear models that yield a functional relationship between predictors and outcome variables. In the machine learning community, the term is also occasionally used loosely to refer to the use of any predictive model that produces a predicted numeric outcome (as opposed to classification methods that predict a binary or categorical outcome).



단순 선형 회귀 > Prediction Versus Explanation (Profiling)

Simple Linear Regression

Prediction Versus Explanation (Profiling)

Historically, a primary use of regression was to illuminate a supposed linear relationship between predictor variables and an outcome variable. The goal has been to understand a relationship and explain it using the data that the regression was fit to.

In this case, the primary focus is on the estimated slope of the regression equation, $\hat{b}$. Economists want to know the relationship between consumer spending and GDP growth. Public health officials might want to understand whether a public information campaign is effective in promoting safe sex practices. In such cases, the focus is not on predicting individual cases but rather on understanding the overall relationship among variables.

With the advent of big data, regression is widely used to form a model to predict individual outcomes for new data (i.e., a predictive model) rather than explain data in hand. In this instance, the main items of interest are the fitted values $\hat{Y}$. In marketing, regression can be used to predict the change in revenue in response to the size of an ad campaign. Universities use regression to predict students' GPA based on their SAT scores.

A regression model that fits the data well is set up such that changes in X lead to changes in Y . However, by itself, the regression equation does not prove the direction of causation. Conclusions about causation must come from a broader understanding about the relationship. For example, a regression equation might show a definite relationship between number of clicks on a web ad and number of conversions. It is our knowledge of the marketing process, not the regression equation, that leads us to the conclusion that clicks on the ad lead to sales, and not vice versa.

Key Ideas

- The regression equation models the relationship between a response variable Y and a predictor variable X as a line.
- A regression model yields fitted values and residuals—predictions of the response and the errors of the predictions.
- Regression models are typically fit by the method of least squares.
- Regression is used both for prediction and explanation.



Multiple Linear Regression

When there are multiple predictors, the equation is simply extended to accommodate them:

$$Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_pX_p + e$$

Instead of a line, we now have a linear model—the relationship between each coefficient and its variable (feature) is linear.

Key Terms for Multiple Linear Regression

Root mean squared error

The square root of the average squared error of the regression (this is the most widely used metric to compare regression models).

Synonym

RMSE

Residual standard error

The same as the root mean squared error, but adjusted for degrees of freedom.

Synonym

RSE

R-squared

The proportion of variance explained by the model, from 0 to 1.

Synonyms

coefficient of determination, R^2

t-statistic

The coefficient for a predictor, divided by the standard error of the coefficient, giving a metric to compare the importance of variables in the model. See “**t-Tests**” on page 110.

Weighted regression

Regression with the records having different weights.

All of the other concepts in simple linear regression, such as fitting by least squares and the definition of fitted values and residuals, extend to the multiple linear regression setting. For example, the fitted values are given by:

$$\hat{Y}_i = \hat{b}_0 + \hat{b}_1X_{1,i} + \hat{b}_2X_{2,i} + \dots + \hat{b}_pX_{p,i}$$



다중 선형 회귀 > Example: King County Housing Data

Multiple Linear Regression

Example: King County Housing Data

An example of using multiple linear regression is in estimating the value of houses. County assessors must estimate the value of a house for the purposes of assessing taxes. Real estate professionals and home buyers consult popular websites such as [Zillow](#) to ascertain a fair price. Here are a few rows of housing data from King County (Seattle), Washington, from the `house` data frame:

```
head(house[, c('AdjSalePrice', 'SqFtTotLiving', 'SqFtLot', 'Bathrooms',
               'Bedrooms', 'BldgGrade')])
Source: local data frame [6 x 6]
```

	AdjSalePrice (dbl)	SqFtTotLiving (int)	SqFtLot (int)	Bathrooms (dbl)	Bedrooms (int)	BldgGrade (int)
1	300805	2400	9373	3.00	6	7
2	1076162	3764	20156	3.75	4	10
3	761805	2060	26036	1.75	4	8
4	442065	3200	8618	3.75	5	7
5	297065	1720	8620	1.75	4	7
6	411781	930	1012	1.50	2	8

The `head` method of pandas data frame lists the top rows:

```
subset = ['AdjSalePrice', 'SqFtTotLiving', 'SqFtLot', 'Bathrooms',
          'Bedrooms', 'BldgGrade']
house[subset].head()
```

The goal is to predict the sales price from the other variables. The `lm` function handles the multiple regression case simply by including more terms on the righthand side of the equation; the argument `na.action=na.omit` causes the model to drop records that have missing values:

```
house_lm <- lm(AdjSalePrice ~ SqFtTotLiving + SqFtLot + Bathrooms +
               Bedrooms + BldgGrade,
               data=house, na.action=na.omit)
```

scikit-learn's `LinearRegression` can be used for multiple linear regression as well:

```
predictors = ['SqFtTotLiving', 'SqFtLot', 'Bathrooms', 'Bedrooms', 'BldgGrade']
outcome = 'AdjSalePrice'
```

```
house_lm = LinearRegression()
house_lm.fit(house[predictors], house[outcome])
```

Printing `house_lm` object produces the following output:

```
house_lm
```

Call:

```
lm(formula = AdjSalePrice ~ SqFtTotLiving + SqFtLot + Bathrooms +
    Bedrooms + BldgGrade, data = house, na.action = na.omit)
```

Coefficients:

(Intercept)	SqFtTotLiving	SqFtLot	Bathrooms
-5.219e+05	2.288e+02	-6.047e-02	-1.944e+04
Bedrooms	BldgGrade		
-4.777e+04	1.061e+05		

For a `LinearRegression` model, intercept and coefficients are the fields `intercept_` and `coef_` of the fitted model:

```
print(f'Intercept: {house_lm.intercept_.3f}')
print('Coefficients:')
for name, coef in zip(predictors, house_lm.coef_):
    print(f' {name}: {coef}')
```

The interpretation of the coefficients is as with simple linear regression: the predicted value $\hat{Y}$ changes by the coefficient b_j for each unit change in X_j assuming all the other variables, X_k for $k \neq j$, remain the same. For example, adding an extra finished square foot to a house increases the estimated value by roughly \$229; adding 1,000 finished square feet implies the value will increase by \$228,800.



다중 선형 회귀 > Assessing the Model

Multiple Linear Regression

Assessing the Model

The most important performance metric from a data science perspective is *root mean squared error*, or *RMSE*. RMSE is the square root of the average squared error in the predicted $\hat{y}_i$ values:

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}}$$

This measures the overall accuracy of the model and is a basis for comparing it to other models (including models fit using machine learning techniques). Similar to RMSE is the *residual standard error*, or *RSE*. In this case we have p predictors, and the RSE is given by:

$$RSE = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{(n - p - 1)}}$$

The only difference is that the denominator is the degrees of freedom, as opposed to number of records (see “**Degrees of Freedom**” on page 116). In practice, for linear regression, the difference between RMSE and RSE is very small, particularly for big data applications.

The `summary` function in *R* computes RSE as well as other metrics for a regression model:

```
summary(house_lm)
```

Call:

```
lm(formula = AdjSalePrice ~ SqFtTotLiving + SqFtLot + Bathrooms +  
    Bedrooms + BldgGrade, data = house, na.action = na.omit)
```

Residuals:

Min	1Q	Median	3Q	Max
-1199479	-118908	-20977	87435	9473035

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-5.219e+05	1.565e+04	-33.342	< 2e-16 ***
SqFtTotLiving	2.288e+02	3.899e+00	58.694	< 2e-16 ***
SqFtLot	-6.047e-02	6.118e-02	-0.988	0.323
Bathrooms	-1.944e+04	3.625e+03	-5.363	8.27e-08 ***
Bedrooms	-4.777e+04	2.490e+03	-19.187	< 2e-16 ***
BldgGrade	1.061e+05	2.396e+03	44.277	< 2e-16 ***

 Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
 Residual standard error: 261300 on 22681 degrees of freedom
 Multiple R-squared: 0.5406, Adjusted R-squared: 0.5405
 F-statistic: 5338 on 5 and 22681 DF, p-value: < 2.2e-16



다중 선형 회귀 > Assessing the Model

Multiple Linear Regression

The `summary` function in R computes RSE as well as other metrics for a regression model:

```
summary(house_lm)

Call:
lm(formula = AdjSalePrice ~ SqFtTotLiving + SqFtLot + Bathrooms +
    Bedrooms + BldgGrade, data = house, na.action = na.omit)

Residuals:
    Min       1Q   Median       3Q      Max
-1199479 -118908  -20977   87435  9473035

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -5.219e+05  1.565e+04 -33.342 < 2e-16 ***
SqFtTotLiving  2.288e+02  3.899e+00  58.694 < 2e-16 ***
SqFtLot       -6.047e-02  6.118e-02  -0.988  0.323
Bathrooms     -1.944e+04  3.625e+03  -5.363 8.27e-08 ***
Bedrooms      -4.777e+04  2.490e+03 -19.187 < 2e-16 ***
BldgGrade      1.061e+05  2.396e+03  44.277 < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Residual standard error: 261300 on 22681 degrees of freedom
Multiple R-squared:  0.5406,    Adjusted R-squared:  0.5405
F-statistic: 5338 on 5 and 22681 DF,  p-value: < 2.2e-16
```

scikit-learn provides a number of metrics for regression and classification. Here, we use `mean_squared_error` to get RMSE and `r2_score` for the coefficient of determination:

```
fitted = house_lm.predict(house[predictors])
RMSE = np.sqrt(mean_squared_error(house[outcome], fitted))
r2 = r2_score(house[outcome], fitted)
print(f'RMSE: {RMSE:.0f}')
print(f'r2: {r2:.4f}')
```

Use `statsmodels` to get a more detailed analysis of the regression model in *Python*:

```
model = sm.OLS(house[outcome], house[predictors].assign(const=1))
results = model.fit()
results.summary()
```

The `pandas` method `assign`, as used here, adds a constant column with value 1 to the predictors. This is required to model the intercept.



다중 선형 회귀 > Assessing the Model

Multiple Linear Regression

Another useful metric that you will see in software output is the *coefficient of determination*, also called the *R-squared* statistic or R^2 . R-squared ranges from 0 to 1 and measures the proportion of variation in the data that is accounted for in the model. It is useful mainly in explanatory uses of regression where you want to assess how well the model fits the data. The formula for R^2 is:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

The denominator is proportional to the variance of Y . The output from *R* also reports an *adjusted R-squared*, which adjusts for the degrees of freedom, effectively penalizing the addition of more predictors to a model. Seldom is this significantly different from *R-squared* in multiple regression with large data sets.

Along with the estimated coefficients, *R* and *statsmodels* report the standard error of the coefficients (SE) and a *t-statistic*:

$$t_b = \frac{\hat{b}}{SE(\hat{b})}$$

The t-statistic—and its mirror image, the p-value—measures the extent to which a coefficient is “statistically significant”—that is, outside the range of what a random chance arrangement of predictor and target variable might produce. The higher the t-statistic (and the lower the p-value), the more significant the predictor. Since parsimony is a valuable model feature, it is useful to have a tool like this to guide choice of variables to include as predictors (see “[Model Selection and Stepwise Regression](#)” on [page 156](#)).



In addition to the t-statistic, *R* and other packages will often report a *p-value* ($\Pr(>|t|)$ in the *R* output) and *F-statistic*. Data scientists do not generally get too involved with the interpretation of these statistics, nor with the issue of statistical significance. Data scientists primarily focus on the t-statistic as a useful guide for whether to include a predictor in a model or not. High t-statistics (which go with p-values near 0) indicate a predictor should be retained in a model, while very low t-statistics indicate a predictor could be dropped. See “[p-Value](#)” on [page 106](#) for more discussion.



다중 선형 회귀 > Cross-Validation

Multiple Linear Regression

Cross-Validation

Classic statistical regression metrics (R^2 , F-statistics, and p-values) are all “in-sample” metrics—they are applied to the same data that was used to fit the model. Intuitively, you can see that it would make a lot of sense to set aside some of the original data, not use it to fit the model, and then apply the model to the set-aside (holdout) data to see how well it does. Normally, you would use a majority of the data to fit the model and use a smaller portion to test the model.

This idea of “out-of-sample” validation is not new, but it did not really take hold until larger data sets became more prevalent; with a small data set, analysts typically want to use all the data and fit the best possible model.

Using a holdout sample, though, leaves you subject to some uncertainty that arises simply from variability in the small holdout sample. How different would the assessment be if you selected a different holdout sample?

Cross-validation extends the idea of a holdout sample to multiple sequential holdout samples. The algorithm for basic *k-fold cross-validation* is as follows:

1. Set aside $1/k$ of the data as a holdout sample.
2. Train the model on the remaining data.
3. Apply (score) the model to the $1/k$ holdout, and record needed model assessment metrics.
4. Restore the first $1/k$ of the data, and set aside the next $1/k$ (excluding any records that got picked the first time).
5. Repeat steps 2 and 3.
6. Repeat until each record has been used in the holdout portion.
7. Average or otherwise combine the model assessment metrics.

The division of the data into the training sample and the holdout sample is also called a *fold*.



다중 선형 회귀 > Model Selection and Stepwise Regression

Multiple Linear Regression

Model Selection and Stepwise Regression

In some problems, many variables could be used as predictors in a regression. For example, to predict house value, additional variables such as the basement size or year built could be used. In R, these are easy to add to the regression equation:

```
house_full <- lm(AdjSalePrice ~ SqFtTotLiving + SqFtLot + Bathrooms +
  Bedrooms + BldgGrade + PropertyType + NbrLivingUnits +
  SqFtFinBasement + YrBuilt + YrRenovated +
  NewConstruction,
  data=house, na.action=na.omit)
```

In Python, we need to convert the categorical and boolean variables into numbers:

```
predictors = ['SqFtTotLiving', 'SqFtLot', 'Bathrooms', 'Bedrooms', 'BldgGrade',
  'PropertyType', 'NbrLivingUnits', 'SqFtFinBasement', 'YrBuilt',
  'YrRenovated', 'NewConstruction']
```

```
X = pd.get_dummies(house[predictors], drop_first=True)
X['NewConstruction'] = [1 if nc else 0 for nc in X['NewConstruction']]
```

```
house_full = sm.OLS(house[outcome], X.assign(const=1))
results = house_full.fit()
results.summary()
```

Adding more variables, however, does not necessarily mean we have a better model. Statisticians use the principle of *Occam's razor* to guide the choice of a model: all things being equal, a simpler model should be used in preference to a more complicated model.

Including additional variables always reduces RMSE and increases R^2 for the training data. Hence, these are not appropriate to help guide the model choice. One approach to including model complexity is to use the adjusted R^2 :

$$R_{adj}^2 = 1 - (1 - R^2) \frac{n - 1}{n - P - 1}$$

Here, n is the number of records and P is the number of variables in the model.

In the 1970s, Hirotugu Akaike, the eminent Japanese statistician, developed a metric called AIC (Akaike's Information Criteria) that penalizes adding terms to a model. In the case of regression, AIC has the form:

$$AIC = 2P + n \log(RSS/n)$$



다중 선형 회귀 > Model Selection and Stepwise Regression

Multiple Linear Regression

where P is the number of variables and n is the number of records. The goal is to find the model that minimizes AIC; models with k more extra variables are penalized by $2k$.



AIC, BIC, and Mallows Cp

The formula for AIC may seem a bit mysterious, but in fact it is based on asymptotic results in information theory. There are several variants to AIC:

AICc

A version of AIC corrected for small sample sizes.

BIC or Bayesian information criteria

Similar to AIC, with a stronger penalty for including additional variables to the model.

Mallows Cp

A variant of AIC developed by Colin Mallows.

These are typically reported as in-sample metrics (i.e., on the training data), and data scientists using holdout data for model assessment do not need to worry about the differences among them or the underlying theory behind them.

How do we find the model that minimizes AIC or maximizes adjusted R^2 ? One way is to search through all possible models, an approach called *all subset regression*. This is computationally expensive and is not feasible for problems with large data and many variables. An attractive alternative is to use *stepwise regression*. It could start with a full model and successively drop variables that don't contribute meaningfully. This is called *backward elimination*. Alternatively one could start with a constant model and successively add variables (*forward selection*). As a third option we can also successively add and drop predictors to find a model that lowers AIC or adjusted R^2 . The MASS in R package by Venebles and Ripley offers a stepwise regression function called stepAIC:

```
library(MASS)
step <- stepAIC(house_full, direction="both")
step
```

Call:

```
lm(formula = AdjSalePrice ~ SqFtTotLiving + Bathrooms + Bedrooms +
    BldgGrade + PropertyType + SqFtFinBasement + YrBuilt, data = house,
    na.action = na.omit)
```

Coefficients:

(Intercept)	SqFtTotLiving
6.179e+06	1.993e+02
Bathrooms	Bedrooms
4.240e+04	-5.195e+04
BldgGrade	PropertyTypeSingle Family
1.372e+05	2.291e+04
PropertyTypeTownhouse	SqFtFinBasement
8.448e+04	7.047e+00
YrBuilt	
-3.565e+03	



다중 선형 회귀 > Model Selection and Stepwise Regression

Multiple Linear Regression

scikit-learn has no implementation for stepwise regression. We implemented functions `stepwise_selection`, `forward_selection`, and `backward_elimination` in our `dmbs` package:

```
y = house[outcome]

def train_model(variables): ❶
    if len(variables) == 0:
        return None
    model = LinearRegression()
    model.fit(X[variables], y)
    return model

def score_model(model, variables): ❷
    if len(variables) == 0:
        return AIC_score(y, [y.mean()] * len(y), model, df=1)
    return AIC_score(y, model.predict(X[variables]), model)

best_model, best_variables = stepwise_selection(X.columns, train_model,
                                                score_model, verbose=True)

print(f'Intercept: {best_model.intercept_:.3f}')
print('Coefficients:')
for name, coef in zip(best_variables, best_model.coef_):
    print(f' {name}: {coef}')
```

- ❶ Define a function that returns a fitted model for a given set of variables.
- ❷ Define a function that returns a score for a given model and set of variables. In this case, we use the `AIC_score` implemented in the `dmbs` package.

The function chose a model in which several variables were dropped from `house_full`: `SqFtLot`, `NbrLivingUnits`, `YrRenovated`, and `NewConstruction`.

Simpler yet are *forward selection* and *backward selection*. In forward selection, you start with no predictors and add them one by one, at each step adding the predictor that has the largest contribution to R^2 , and stopping when the contribution is no longer statistically significant. In backward selection, or *backward elimination*, you start with the full model and take away predictors that are not statistically significant until you are left with a model in which all predictors are statistically significant.

Penalized regression is similar in spirit to AIC. Instead of explicitly searching through a discrete set of models, the model-fitting equation incorporates a constraint that penalizes the model for too many variables (parameters). Rather than eliminating predictor variables entirely—as with stepwise, forward, and backward selection—penalized regression applies the penalty by reducing coefficients, in some cases to near zero. Common penalized regression methods are *ridge regression* and *lasso regression*.

Stepwise regression and all subset regression are *in-sample* methods to assess and tune models. This means the model selection is possibly subject to overfitting (fitting the noise in the data) and may not perform as well when applied to new data. One common approach to avoid this is to use cross-validation to validate the models. In linear regression, overfitting is typically not a major issue, due to the simple (linear) global structure imposed on the data. For more sophisticated types of models, particularly iterative procedures that respond to local data structure, cross-validation is a very important tool; see “[Cross-Validation](#)” on page 155 for details.



다중 선형 회귀 > Weighted Regression

Multiple Linear Regression

Weighted Regression

Weighted regression is used by statisticians for a variety of purposes; in particular, it is important for analysis of complex surveys. Data scientists may find weighted regression useful in two cases:

- Inverse-variance weighting when different observations have been measured with different precision; the higher variance ones receiving lower weights.
- Analysis of data where rows represent multiple cases; the weight variable encodes how many original observations each row represents.

For example, with the housing data, older sales are less reliable than more recent sales. Using the DocumentDate to determine the year of the sale, we can compute a Weight as the number of years since 2005 (the beginning of the data):

R

```
library(lubridate)
house$Year = year(house$DocumentDate)
house$Weight = house$Year - 2005
```

Python

```
house['Year'] = [int(date.split('-')[0]) for date in house.DocumentDate]
house['Weight'] = house.Year - 2005
```

We can compute a weighted regression with the `lm` function using the `weight` argument:

```
house_wt <- lm(AdjSalePrice ~ SqFtTotLiving + SqFtLot + Bathrooms +
               Bedrooms + BldgGrade,
               data=house, weight=Weight)
round(cbind(house_lm=house_lm$coefficients,
            house_wt=house_wt$coefficients), digits=3)
```

	house_lm	house_wt
(Intercept)	-521871.368	-584189.329
SqFtTotLiving	228.831	245.024
SqFtLot	-0.060	-0.292
Bathrooms	-19442.840	-26085.970
Bedrooms	-47769.955	-53608.876
BldgGrade	106106.963	115242.435

The coefficients in the weighted regression are slightly different from the original regression.

Most models in `scikit-learn` accept weights as the keyword argument `sample_weight` in the call of the fit method:

```
predictors = ['SqFtTotLiving', 'SqFtLot', 'Bathrooms', 'Bedrooms', 'BldgGrade']
outcome = 'AdjSalePrice'

house_wt = LinearRegression()
house_wt.fit(house[predictors], house[outcome], sample_weight=house.Weight)
```



다중 선형 회귀 > *Weighted Regression*

Multiple Linear Regression

Key Ideas

- Multiple linear regression models the relationship between a response variable Y and multiple predictor variables $X_1, \dots, X_p$.
- The most important metrics to evaluate a model are root mean squared error (RMSE) and R-squared (R^2).
- The standard error of the coefficients can be used to measure the reliability of a variable's contribution to a model.
- Stepwise regression is a way to automatically determine which variables should be included in the model.
- Weighted regression is used to give certain records more or less weight in fitting the equation.



회귀 기반 예측

Prediction Using Regression

Prediction Using Regression

The primary purpose of regression in data science is prediction. This is useful to keep in mind, since regression, being an old and established statistical method, comes with baggage that is more relevant to its traditional role as a tool for explanatory modeling than to prediction.

Key Terms for Prediction Using Regression

Prediction interval

An uncertainty interval around an individual predicted value.

Extrapolation

Extension of a model beyond the range of the data used to fit it.



회귀 기반 예측 > *The Dangers of Extrapolation*

Prediction Using Regression

The Dangers of Extrapolation

Regression models should not be used to extrapolate beyond the range of the data (leaving aside the use of regression for time series forecasting.). The model is valid only for predictor values for which the data has sufficient values (even in the case that sufficient data is available, there could be other problems—see “**Regression Diagnostics**” on page 176). As an extreme case, suppose `model_lm` is used to predict the value of a 5,000-square-foot empty lot. In such a case, all the predictors related to the building would have a value of 0, and the regression equation would yield an absurd prediction of $-521,900 + 5,000 \times -.0605 = -\$522,202$. Why did this happen? The data contains only parcels with buildings—there are no records corresponding to vacant land. Consequently, the model has no information to tell it how to predict the sales price for vacant land.



회귀 기반 예측 > Confidence and Prediction Intervals

Prediction Using Regression

Confidence and Prediction Intervals

Much of statistics involves understanding and measuring variability (uncertainty). The t-statistics and p-values reported in regression output deal with this in a formal way, which is sometimes useful for variable selection (see “Assessing the Model” on page 153). More useful metrics are confidence intervals, which are uncertainty intervals placed around regression coefficients and predictions. An easy way to understand this is via the bootstrap (see “The Bootstrap” on page 61 for more details about the general bootstrap procedure). The most common regression confidence intervals encountered in software output are those for regression parameters (coefficients). Here is a bootstrap algorithm for generating confidence intervals for regression parameters (coefficients) for a data set with P predictors and n records (rows):

1. Consider each row (including outcome variable) as a single “ticket” and place all the n tickets in a box.
2. Draw a ticket at random, record the values, and replace it in the box.
3. Repeat step 2 n times; you now have one bootstrap resample.
4. Fit a regression to the bootstrap sample, and record the estimated coefficients.
5. Repeat steps 2 through 4, say, 1,000 times.
6. You now have 1,000 bootstrap values for each coefficient; find the appropriate percentiles for each one (e.g., 5th and 95th for a 90% confidence interval).

You can use the `Boot` function in *R* to generate actual bootstrap confidence intervals for the coefficients, or you can simply use the formula-based intervals that are a routine *R* output. The conceptual meaning and interpretation are the same, and not of central importance to data scientists, because they concern the regression coefficients. Of greater interest to data scientists are intervals around predicted y values ($\hat{Y}_i$). The uncertainty around $\hat{Y}_i$ comes from two sources:

- Uncertainty about what the relevant predictor variables and their coefficients are (see the preceding bootstrap algorithm)
- Additional error inherent in individual data points

The individual data point error can be thought of as follows: even if we knew for certain what the regression equation was (e.g., if we had a huge number of records to fit it), the *actual* outcome values for a given set of predictor values will vary. For example, several houses—each with 8 rooms, a 6,500-square-foot lot, 3 bathrooms, and a basement—might have different values. We can model this individual error with the residuals from the fitted values. The bootstrap algorithm for modeling both the regression model error and the individual data point error would look as follows:

1. Take a bootstrap sample from the data (spelled out in greater detail earlier).
2. Fit the regression, and predict the new value.
3. Take a single residual at random from the original regression fit, add it to the predicted value, and record the result.
4. Repeat steps 1 through 3, say, 1,000 times.
5. Find the 2.5th and the 97.5th percentiles of the results.



회귀 기반 예측 > *Confidence and Prediction Intervals*

Prediction Using Regression

Key Ideas

- Extrapolation beyond the range of the data can lead to error.
- Confidence intervals quantify uncertainty around regression coefficients.
- Prediction intervals quantify uncertainty in individual predictions.
- Most software, R included, will produce prediction and confidence intervals in default or specified output, using formulas.
- The bootstrap can also be used to produce prediction and confidence intervals; the interpretation and idea are the same.



Prediction Interval or Confidence Interval?

A prediction interval pertains to uncertainty around a single value, while a confidence interval pertains to a mean or other statistic calculated from multiple values. Thus, a prediction interval will typically be much wider than a confidence interval for the same value. We model this individual value error in the bootstrap model by selecting an individual residual to tack on to the predicted value. Which should you use? That depends on the context and the purpose of the analysis, but, in general, data scientists are interested in specific individual predictions, so a prediction interval would be more appropriate. Using a confidence interval when you should be using a prediction interval will greatly underestimate the uncertainty in a given predicted value.



Factor Variables in Regression

Factor variables, also termed *categorical* variables, take on a limited number of discrete values. For example, a loan purpose can be “debt consolidation,” “wedding,” “car,” and so on. The binary (yes/no) variable, also called an *indicator* variable, is a special case of a factor variable. Regression requires numerical inputs, so factor variables need to be recoded to use in the model. The most common approach is to convert a variable into a set of binary *dummy* variables.

Key Terms for Factor Variables

Dummy variables

Binary 0–1 variables derived by recoding factor data for use in regression and other models.

Reference coding

The most common type of coding used by statisticians, in which one level of a factor is used as a reference and other factors are compared to that level.

Synonym

treatment coding

One hot encoder

A common type of coding used in the machine learning community in which all factor levels are retained. While useful for certain machine learning algorithms, this approach is not appropriate for multiple linear regression.

Deviation coding

A type of coding that compares each level against the overall mean as opposed to the reference level.

Synonym

sum contrasts

AI Factor 변수 > Dummy Variables Representation

Factor Variables in Regression

Dummy Variables Representation

In the King County housing data, there is a factor variable for the property type; a small subset of six records is shown below:

R:

```
head(house[, 'PropertyType'])
Source: local data frame [6 x 1]
```

```
PropertyType
(fctr)
1    Multiplex
2 Single Family
3 Single Family
4 Single Family
5 Single Family
6    Townhouse
```

Python:

```
house.PropertyType.head()
```

There are three possible values: Multiplex, Single Family, and Townhouse. To use this factor variable, we need to convert it to a set of binary variables. We do this by creating a binary variable for each possible value of the factor variable. To do this in R, we use the `model.matrix` function:³

```
prop_type_dummies <- model.matrix(~PropertyType -1, data=house)
head(prop_type_dummies)
PropertyTypeMultiplex PropertyTypeSingle Family PropertyTypeTownhouse
1          1          0          0
2          0          1          0
3          0          1          0
4          0          1          0
5          0          1          0
6          0          0          1
```

The function `model.matrix` converts a data frame into a matrix suitable to a linear model. The factor variable `PropertyType`, which has three distinct levels, is represented as a matrix with three columns. In the machine learning community, this representation is referred to as *one hot encoding* (see [“One Hot Encoder” on page 242](#)).

In Python, we can convert categorical variables to dummies using the pandas method `get_dummies`:

```
pd.get_dummies(house['PropertyType']).head() ❶
pd.get_dummies(house['PropertyType'], drop_first=True).head() ❷
```

- ❶ By default, returns one hot encoding of the categorical variable.
- ❷ The keyword argument `drop_first` will return $P - 1$ columns. Use this to avoid the problem of multicollinearity.

In certain machine learning algorithms, such as nearest neighbors and tree models, one hot encoding is the standard way to represent factor variables (for example, see [“Tree Models” on page 249](#)).

In the regression setting, a factor variable with P distinct levels is usually represented by a matrix with only $P - 1$ columns. This is because a regression model typically includes an intercept term. With an intercept, once you have defined the values for $P - 1$ binaries, the value for the P th is known and could be considered redundant. Adding the P th column will cause a multicollinearity error (see [“Multicollinearity” on page 172](#)).



Factor 변수 > Dummy Variables Representation

Factor Variables in Regression

The default representation in R is to use the first factor level as a *reference* and interpret the remaining levels relative to that factor:

```
lm(AdjSalePrice ~ SqFtTotLiving + SqFtLot + Bathrooms +  
  Bedrooms + BldgGrade + PropertyType, data=house)
```

Call:

```
lm(formula = AdjSalePrice ~ SqFtTotLiving + SqFtLot + Bathrooms +  
  Bedrooms + BldgGrade + PropertyType, data = house)
```

Coefficients:

(Intercept)	SqFtTotLiving
-4.468e+05	2.234e+02
SqFtLot	Bathrooms
-7.037e-02	-1.598e+04
Bedrooms	BldgGrade
-5.089e+04	1.094e+05
PropertyTypeSingle Family	PropertyTypeTownhouse
-8.468e+04	-1.151e+05

The method `get_dummies` takes the optional keyword argument `drop_first` to exclude the first factor as *reference*:

```
predictors = ['SqFtTotLiving', 'SqFtLot', 'Bathrooms', 'Bedrooms',  
  'BldgGrade', 'PropertyType']
```

```
X = pd.get_dummies(house[predictors], drop_first=True)
```

```
house_lm_factor = LinearRegression()  
house_lm_factor.fit(X, house[outcome])
```

```
print(f'Intercept: {house_lm_factor.intercept_:.3f}')  
print('Coefficients:')  
for name, coef in zip(X.columns, house_lm_factor.coef_):  
  print(f' {name}: {coef}')
```

The output from the R regression shows two coefficients corresponding to Property Type: `PropertyTypeSingle Family` and `PropertyTypeTownhouse`. There is no coefficient of `Multiplex` since it is implicitly defined when `PropertyTypeSingle Family == 0` and `PropertyTypeTownhouse == 0`. The coefficients are interpreted as relative to `Multiplex`, so a home that is `Single Family` is worth almost \$85,000 less, and a home that is `Townhouse` is worth over \$150,000 less.⁴



Different Factor Codings

There are several different ways to encode factor variables, known as *contrast coding* systems. For example, *deviation coding*, also known as *sum contrasts*, compares each level against the overall mean. Another contrast is *polynomial coding*, which is appropriate for ordered factors; see the section “[Ordered Factor Variables](#)” on [page 169](#). With the exception of ordered factors, data scientists will generally not encounter any type of coding besides reference coding or one hot encoder.



Factor 변수 > Factor Variables with Many Levels

Factor Variables in Regression

Factor Variables with Many Levels

Some factor variables can produce a huge number of binary dummies—zip codes are a factor variable, and there are 43,000 zip codes in the US. In such cases, it is useful to explore the data, and the relationships between predictor variables and the outcome, to determine whether useful information is contained in the categories. If so, you must further decide whether it is useful to retain all factors, or whether the levels should be consolidated.

In King County, there are 80 zip codes with a house sale:

```
table(house$ZipCode)
```

```

98001 98002 98003 98004 98005 98006 98007 98008 98010 98011 98014 98019
 358   180   241   293   133   460   112   291    56   163    85   242
98022 98023 98024 98027 98028 98029 98030 98031 98032 98033 98034 98038
 188   455    31   366   252   475   263   308   121   517   575   788
98039 98040 98042 98043 98045 98047 98050 98051 98052 98053 98055 98056
  47   244   641    1   222   48    7   32   614   499   332   402
98057 98058 98059 98065 98068 98070 98072 98074 98075 98077 98092 98102
   4   420   513   430    1    89   245   502   388   204   289   106
98103 98105 98106 98107 98108 98109 98112 98113 98115 98116 98117 98118
 671   313   361   296   155   149   357    1   620   364   619   492
98119 98122 98125 98126 98133 98136 98144 98146 98148 98155 98166 98168
 260   380   409   473   465   310   332   287   40   358   193   332
98177 98178 98188 98198 98199 98224 98288 98354
 216   266   101   225   393    3    4    9

```

The `value_counts` method of pandas data frames returns the same information:

```
pd.DataFrame(house['ZipCode'].value_counts()).transpose()
```

ZipCode is an important variable, since it is a proxy for the effect of location on the value of a house. Including all levels requires 79 coefficients corresponding to 79 degrees of freedom. The original model `house_lm` has only 5 degrees of freedom; see [“Assessing the Model” on page 153](#). Moreover, several zip codes have only one sale. In some problems, you can consolidate a zip code using the first two or three digits, corresponding to a submetropolitan geographic region. For King County, almost all of the sales occur in 980xx or 981xx, so this doesn’t help.

An alternative approach is to group the zip codes according to another variable, such as sale price. Even better is to form zip code groups using the residuals from an initial model. The following `dplyr` code in *R* consolidates the 80 zip codes into five groups based on the median of the residual from the `house_lm` regression:

```

zip_groups <- house %>%
  mutate(resid = residuals(house_lm)) %>%
  group_by(ZipCode) %>%
  summarize(med_resid = median(resid),
            cnt = n()) %>%
  arrange(med_resid) %>%
  mutate(cum_cnt = cumsum(cnt),
         ZipGroup = ntile(cum_cnt, 5))
house <- house %>%
  left_join(select(zip_groups, ZipCode, ZipGroup), by='ZipCode')

```



Factor 변수 > Factor Variables with Many Levels

Factor Variables in Regression

The median residual is computed for each zip, and the `ntile` function is used to split the zip codes, sorted by the median, into five groups. See “[Confounding Variables](#)” on [page 172](#) for an example of how this is used as a term in a regression improving upon the original fit.

In *Python* we can calculate this information as follows:

```
zip_groups = pd.DataFrame([
    *pd.DataFrame({
        'ZipCode': house['ZipCode'],
        'residual' : house[outcome] - house_lm.predict(house[predictors]),
    })
    .groupby(['ZipCode'])
    .apply(lambda x: {
        'ZipCode': x.iloc[0,0],
        'count': len(x),
        'median_residual': x.residual.median()
    })
]).sort_values('median_residual')
zip_groups['cum_count'] = np.cumsum(zip_groups['count'])
zip_groups['ZipGroup'] = pd.qcut(zip_groups['cum_count'], 5, labels=False,
                                retbins=False)

to_join = zip_groups[['ZipCode', 'ZipGroup']].set_index('ZipCode')
house = house.join(to_join, on='ZipCode')
house['ZipGroup'] = house['ZipGroup'].astype('category')
```

The concept of using the residuals to help guide the regression fitting is a fundamental step in the modeling process; see “[Regression Diagnostics](#)” on [page 176](#).



Factor 변수 > Ordered Factor Variables

Factor Variables in Regression

Ordered Factor Variables

Some factor variables reflect levels of a factor; these are termed *ordered factor variables* or *ordered categorical variables*. For example, the loan grade could be A, B, C, and so on—each grade carries more risk than the prior grade. Often, ordered factor variables can be converted to numerical values and used as is. For example, the variable BldgGrade is an ordered factor variable. Several of the types of grades are shown in Table 4-1. While the grades have specific meaning, the numeric value is ordered from low to high, corresponding to higher-grade homes. With the regression model house_lm, fit in “Multiple Linear Regression” on page 150, BldgGrade was treated as a numeric variable.

Table 4-1. Building grades and numeric equivalents

Value	Description
1	Cabin
2	Substandard
5	Fair
10	Very good
12	Luxury
13	Mansion

Treating ordered factors as a numeric variable preserves the information contained in the ordering that would be lost if it were converted to a factor.

Key Ideas

- Factor variables need to be converted into numeric variables for use in a regression.
- The most common method to encode a factor variable with P distinct values is to represent them using P – 1 dummy variables.
- A factor variable with many levels, even in very big data sets, may need to be consolidated into a variable with fewer levels.
- Some factors have levels that are ordered and can be represented as a single numeric variable.



기말 프로젝트 관련 공지

Info about Final Term Project

기말 프로젝트 안내

1. 팀 구성 마감
 - 마감일: 2025년 05월 09일 토요일 밤 12:00까지
2. 기획서 제출
 - 제출 마감: 2025년 05월 15일 금요일 밤 12:00까지
3. 기획서 예시
 - 예시 문서 링크: [기획서 예시 보러가기](#)



다음 주 배울 내용

Contents for Next Week

1부: 회귀 및 예측

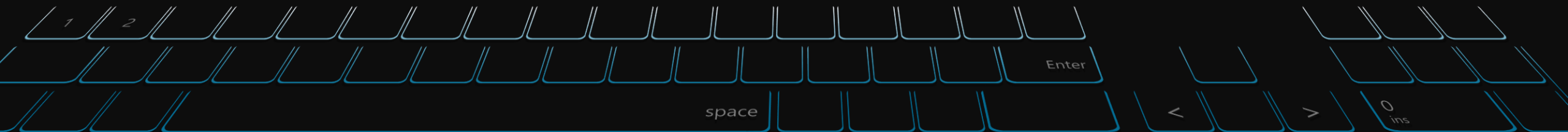
- 회귀방정식 해석
- 회귀분석 진단하기
- 회귀분석 (다항 / 스플라인)
- etc

2부: 분류

- 나이브 베이즈
- 분류분석
- 로지스틱 회귀
- etc

Q&A

질의응답 (5분)



끝. 감사합니다.

수업 들느라 수고하셨습니다.

